COMPUTING MINIMUM REAL RANK OF UNAMBIGUOUS AUTOMATA IN NC2 AND CONSTRUCTING A MINIMUM-RANK MATRIX

STEFAN KIEFER, ANDREW RYZHIKOV

Editorial scope. One UFA/zero-one-matrix-monoid minimum real rank algorithm group: NC2 computation, O(mn4)-time O(n2)-space rank computation and O(mn4)-time O(n3)-space minimum-rank matrix construction. Preserve3 original Proof sketch headings and actual implementation/cost arguments without formal-certification claims. Total-DFA specialization, SLP consequence and algebraic synchronization criterion retained as same-source context, not additional targets.

Allowed standard prerequisites (editorial).

- Author-cited standard foundations. All local definitions, main supporting constructions and complete source bibliography are retained; cited standard foundations remain original references, with no new proof or assumption.

Original publication credits. We thank the anonymous reviewers of the preliminary version for their helpful comments that improved the presentation of the paper. Andrew Ryzhikov is supported by Polish National Science Centre SONATA BIS-12 grant number 2022/46/E/ST6/00230.

Original abstract. A zero-one matrix is a matrix with entries from $\{0, 1\}$. We study monoids containing only such matrices. A finite set of zero-one matrices generating such a monoid can be seen as the matrix representation of an unambiguous finite automaton, an important generalisation of deterministic finite automata which shares many of their good properties.

Let $\mathcal{A}$ be a finite set of $n \times n$ zero-one matrices generating a monoid of zero-one matrices, and m be the cardinality of $\mathcal{A}$. We study the computational complexity of computing the minimum rank of a matrix in the monoid generated by $\mathcal{A}$. By using linear-algebraic techniques, we show that this problem is in NC and can be solved in $\mathcal{O}(mn^4)$ time and $\mathcal{O}(n^2)$ space. We also provide a combinatorial algorithm finding a matrix of minimum rank in $\mathcal{O}(mn^4)$ time and $\mathcal{O}(n^3)$ space. As a byproduct, we show a very weak version of a generalisation of the Černý conjecture: there always exists a straight line program of size $\mathcal{O}(n^2)$ describing a product resulting in a matrix of minimum rank.

For the special case corresponding to total DFAs (that is, for the case where all matrices have exactly one 1 in each row), the minimum rank is the size of the smallest image of the set of all states under the action of a word. Our combinatorial algorithm finds a matrix of minimum rank in time $\mathcal{O}(n^3 + mn^2)$ in this case.

1. Introduction

Matrix monoids are a rich and versatile object naturally appearing in formal verification, program analysis, dynamical systems and weighted automata. However, many of their properties are in general undecidable. One such example is the well-studied matrix mortality problem. Given a finite set $\mathcal{A}$ of $n \times n$ matrices, it asks if the monoid generated by $\mathcal{A}$ (that is, the set of all products of matrices from $\mathcal{A}$) contains the zero matrix. This problem is undecidable already for 3×3 integer matrices [Pat70], and was studied for several decidable special cases, see e.g. [CHHN14, BPS21, Ryz24].

Date: arxiv:2511.09703v5; CC BY 4.0.

Even if $\mathcal{A}$ is a set of two zero-one matrices (that is, matrices with entries in $\{0,1\}$), matrix mortality is PSPACE-complete [Ryz24]. Thus, to make it tractable, one has to further restrict the problem. In this paper, we consider the case where the whole monoid generated by $\mathcal{A}$ consists of zero-one matrices; matrix mortality then becomes decidable in polynomial time [KM21]. We call such monoids *zero-one matrix monoids*. Intuitively, when multiplying any two matrices from such a monoid, we never get $1+1$ as a subexpression. Zero-one matrix monoids have a rich structure while still admitting good algorithmic properties. They correspond precisely to unambiguous finite automata, and find applications in formal verification [BKK+23], variable-length codes [BPR10] and symbolic dynamics [LM21]. They are also an interesting special case of finite monoids of rational matrices (studied in, e.g., [MS77, Jac77, AS09, BHK+20]), monoids of nonnegative matrices (studied in, e.g., [Pro21, BJO15, WZ23a, GGJ18]), and, in the case where they do not contain the zero matrix, of matrix monoids with constant spectral radius [PV17b].

In this paper, we consider a problem that can be seen as a natural generalisation of matrix mortality: given a finite set $\mathcal{A}$ generating a zero-one matrix monoid, find the minimum real rank of a matrix in this monoid. By the real rank of a matrix we mean the dimension of the subspace generated by its columns over the reals. Clearly, this rank is zero if and only if the monoid contains the zero matrix. The minimum real rank of a matrix in a zero-one matrix monoid is a much more tractable problem than deciding other similar properties: for example, checking if a zero-one matrix monoid contains a matrix of a given real rank was shown to be NP-hard[1] [GK95], and checking if it contains a given matrix is a classical PSPACE-complete problem [Koz77].

An important motivation for considering the minimum rank in a zero-one matrix monoid comes from a probabilistic perspective. Let $\mathcal{A}$ be a finite set of matrices, and consider a product of ℓ matrices where at each position a matrix from $\mathcal{A}$ is chosen uniformly at random. It is easy to see that when ℓ tends to infinity, the probability that the rank of this product is equal to the minimum rank of matrices from the monoid generated by $\mathcal{A}$ converges to one. Thus, this minimum rank characterises the most likely eventual behaviour of a linear dynamical system corresponding to $\mathcal{A}$.

The goals of our paper are as follows.

- We present efficient algorithms for analysing monoids of zero-one matrices and unambiguous finite automata.
- To obtain these algorithms, we provide new structural and algebraic properties of such monoids and automata that might be interesting on their own.
- In particular, we provide an algebraic obstacle for an unambiguous finite automaton to be synchronising, similar to the known result for deterministic finite automata.
- We thus strengthen the connections between the areas of synchronising automata, weighted automata and matrix semigroups by transferring methods and tools between them.
- Finally, we highlight open problems in the intersection of these areas.

2. Existing results and our contributions

Throughout the paper, we always assume that matrix monoids are defined by sets of generators, and all matrices are square zero-one unless stated otherwise.

2.1. **Total DFAs.**

An $n \times n$ zero-one matrix with exactly one 1 in every row can be equivalently seen as a transformation of a set Q of size n. A set of such matrices generates a zero-one matrix monoid, and can be seen as a total deterministic finite (semi-)automaton[2] (total DFA) $\mathcal{A} = (Q, \Sigma, \delta)$. Here, Σ

[1] In fact, it is PSPACE-complete, which follows directly from [Ber14, Theorem 3]: add a fresh state and define all yet undefined transitions to lead to this state.

[2] In this paper, all automata are semi-automata, meaning that they do not have any initial or accepting states, and thus do not recognise any languages. Following the usual conventions (as in, e.g., [BPR10]), we omit "semi-", in particular because it would make abbreviations like DFA less recognisable. Total DFAs are often called complete, but we prefer the term "total" both to avoid the clash of terminology with complete UFAs and because of the obvious connection with total functions.

is a finite alphabet whose letters correspond to the generating matrices, and $\delta : Q \times \Sigma \to Q$ is the transition function defined in such a way that for each $a \in \Sigma$, $\delta(_, a)$ is the transformation of Q induced in the natural way by the matrix corresponding to a. Thus, words over Σ correspond to products of the generating matrices.

The *rank* of a word w in $\mathcal{A}$ is the size of the image of Q under the transformation corresponding to w. Equivalently, it is the real rank of the matrix corresponding to w. The *rank* of a total DFA is the minimum among the ranks of all its words. This concept was studied from the perspectives of automata theory [Rys92, KRV19] and the theory of transformation semigroups [SY10, Kar17]. It is the subject of the rank conjecture (called the Černý-Pin conjecture in [Rys92]), which states that every total DFA of rank r admits a word of rank r having length at most $(n-r)^2$. The Černý conjecture, one of the oldest open problems in combinatorial automata theory [Vol22], is a special case with $r = 1$. We refer to surveys [Vol08a, BP16b, KRV19, Vol22] for the vast literature on the Černý conjecture. Underlying digraphs of total DFAs of a given rank were studied in [BF11, BP16b] in the context of the road colouring problem.

The rank of an n-state total DFA over an alphabet of size m can be found in $\mathcal{O}(m^4n^4)$ time [Rys92, Theorem 1]. In contrast, for any fixed $r \geq 2$, the problem of checking if a total DFA admits a word of rank r is NP-hard [GK95]. Checking if an n-state total DFA over an alphabet of size m has rank one is NL-complete [HJ18, Vol22], and can be done in $\mathcal{O}(mn^2)$ time [Epp90, Vol08a]. For each total DFA of rank r, there exists a word of rank r of length at most $\frac{(n-r)^3}{6} + \mathcal{O}((n-r)^2)$ [KRS87], and if $r = 1$, finding a word of rank one can be done in $\mathcal{O}(n^3 + mn^2)$ time and $\mathcal{O}(n^2)$ space [Epp90].

2.2. **Unambiguous finite automata.** Generalising the case of total DFAs, a set $\mathcal{A}$ of $n \times n$ zero-one matrices generating a zero-one matrix monoid can be equivalently seen as an unambiguous nondeterministic finite (semi-)automaton (UFA). Let $Q = \{q_1, \ldots, q_n\}$ be its set of states. To each matrix in $\mathcal{A}$ we again associate a letter in the alphabet Σ, and the transition relation $\Delta \subseteq Q \times \Sigma \times Q$ is defined so that $(q_i, a, q_j) \in \Delta$ if and only if the entry (i, j) in the matrix corresponding to a is equal to one. Just as in the total DFA case, words over Σ naturally correspond to products of matrices from $\mathcal{A}$.

The obtained NFA then has the property that is sometimes called *diamond-free*: for every two states p, q and every word w, there is at most one path from p to q labelled by w. A simple reachability argument shows that the length of a shortest word labelling two such paths, if it exists, is at most quadratic in the dimension of the matrices. Hence, deciding whether an NFA is a UFA (and thus whether a set of zero-one matrices generates a zero-one monoid) is in $\mathsf{coNL} = \mathsf{NL}$. It is actually NL-complete as described in the next subsection.

A UFA is called complete if it does not admit a word whose matrix is the zero matrix. For an n-state UFA the length of such a word if it exists is at most n^5 [KM21]. The best known lower bound is quadratic in n, and is achieved by a series of DFAs [Rys97]. For UFAs, the quadratic upper bound was conjectured to be tight [Rys92, Conjecture 2]. Checking if a UFA is complete can be done in NC^2 [KM21].

The *real rank* of a UFA is the minimum among the real ranks of the matrices corresponding to words. It was shown in [Car88] that for an n-state UFA of real rank $r \geq 1$ there always exists a word of minimum rank of length $\mathcal{O}(rn^3)$. For n-state strongly connected Eulerian UFAs of rank one, a subclass with remarkably nice properties, there always exists a word of length at most $(n-1)^2$ of rank one [CD09, Corollary 4]. All mentioned constructions also provide polynomial time algorithms that construct words with the required properties (in particular, with a length within the stated bounds).

2.3. **Applications to variable-length codes.** A *variable-length code* (or simply a *code*) is a set X of finite words over an alphabet Σ such that every finite word over Σ has at most one factorisation over X. In other words, a code is a basis of a free submonoid of Σ^*.

The definitions of both UFAs and codes rely, intuitively, on the uniqueness of certain representations. In fact, UFAs and codes are tightly related. Let us illustrate this relationship. If the cardinality of a code X is finite, one can construct its flower automaton, which is a UFA with a chosen state s such that, for each word from X, there is a separate cycle containing s and labelled by this word, see Figure 1 (left) for an example. More generally, codes that are regular languages correspond precisely to strongly connected UFAs in a similar way, see [BPR10, Chapter 4] for the details.

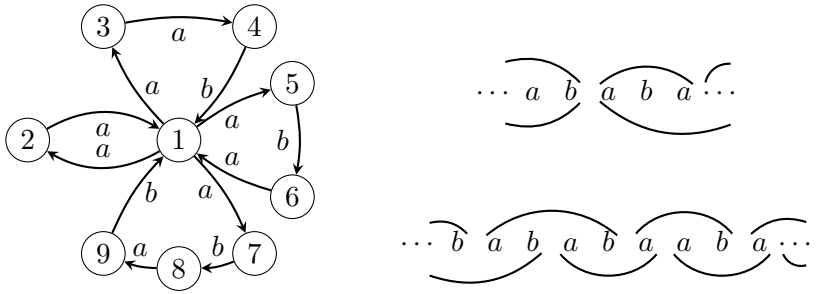


FIGURE 1. The flower automaton of the code $X = \{aa, aab, aba, abab\}$ (left), two adjacent interpretations of *ababa* over X (top right), and two disjoint interpretations of *bababaaba* over X (bottom right). Note that this code is not complete, but still illustrates all the discussed properties.

A useful application is the fact that deciding if a set of zero-one matrices generates a zero-one monoid is NL-hard. Indeed, a finite set of words is a code if and only if its flower automaton is unambiguous [BPR10]. Deciding if a finite set of words is a code is NL-complete [Ryt86], and the flower automaton can be constructed in AC^0.

A code X over Σ is called *complete* if every word over Σ is a factor of a concatenation of codewords, that is, for every word $w \in \Sigma^*$ there exist $u, v \in \Sigma^*$ with $uwv \in X^*$. A code that is a regular language is complete if and only if the corresponding UFA is complete [BPR10]. For complete codes that are regular languages, the real rank of the corresponding UFA is equal to a natural and important parameter called the degree of a code [BPR10, Proposition 9.6.1].

2.4. **The degree of a code.** Let us first explain the intuition behind the notion of degree. For each word w we can consider all possible factorisations over X of all its extensions $uwv \in X^*$ with $u, v \in \Sigma^*$, called *interpretations* of w. Two such interpretations either match in at least one position (as in Figure 1 (top right) between the second and the third letter), or do not match in any position (as in Figure 1 (bottom right)), in which case they are called *disjoint*. The degree of a word is the number of pairwise disjoint interpretations of this word. The degree of a code X is the minimum nonzero degree of all words $w \in \Sigma^*$.

Formally, an *interpretation* of a word w over a code X is a triple (d, x, g) such that d is a suffix of a word from X, $x \in X^*$, and g is a prefix of a word from X. Two interpretations (d, x, g) and (d', x', g') of w are said to be *adjacent* if there exist $y, z, y', z' \in X^*$ with $x = yz, x' = y'z', dy = d'y', zg = z'g'$. Two interpretations are said to be *disjoint* if they are not adjacent. See Figure 1 for an example. The *degree* of a word is the number of pairwise disjoint interpretations of this word. The *degree* of a code X is the minimum nonzero degree of all words $w \in \Sigma^*$. For codes that are regular languages, the degree is equal to the minimum nonzero rank of the corresponding UFA [BPR10, Proposition 9.6.1]. In particular, if there exists a word $w \in X^*$ of degree 1 (called a synchronising word) for a code X, then any sequence $xwwy \in X^*$ of codewords with $x, y \in X^*$ is guaranteed to split into $xw, wy \in X^*$, thus allowing independent decoding of the two halves.

A particularly important case is when a complete code has degree one. Then there exists a word $w \in X^*$ (called a synchronising word) such that for any concatenation of codewords $uwwv \in X^*$ with $u, v \in \Sigma^*$ we have $uw, wv \in X^*$. Intuitively, this means that the two halves uw and wv can be decoded separately and independently.

2.5. **Computational complexity classes.** In this paper, we characterise the computational complexity of problems by showing that they belong to the classes $\mathsf{NL} \subseteq \mathsf{NC}^2 \subseteq \mathsf{NC} \subseteq \mathsf{P}$, see [AB09, Gol08] for their formal definitions. NL is the class of problems solvable in nondeterministic logarithmic time. NC^k is the class of problems solvable by $\mathcal{O}((\log n)^k)$-depth polynomial-size bounded fan-in Boolean circuits, and NC is the union of these classes for all $k \geq 1$. The class NC represents problems that have efficient parallel algorithms, and is a subclass of problems solvable in polylogarithmic space [AB09]. Intuitively, NC is the class of problems that can be solved using local computations, as opposed to P-complete problems, which are inherently sequential and thus require storing the entire structure in the memory unless $\mathsf{NC} = \mathsf{P}$. An important property of NC is that problems from this class can be used for designing PSPACE algorithms as discussed, e.g., in the beginning of [JJUW11, Section 3]. Namely, the composition of a PSPACE-transducer and an NC-algorithm is a PSPACE-algorithm [Bor77], despite the fact that the output of a PSPACE-transduction can have exponential size. This is not necessarily true when an NC-algorithm is replaced by an arbitrary polynomial time algorithm. In the context of formal verification, this compositional approach is used, e.g., in [BKK+23].

NC^2 is an especially important class in computational algebra. To quote [Gol08, page 468], "NC^2 is the habitat of most natural problems in linear algebra". Indeed, matrix multiplication, computing the determinant, inverse and rank of a matrix belong to NC^2 [BvzGH82, Coo85, Ber84, Csa76].

2.6. **Our contributions.** The known results about reachability properties of zero-one matrix monoids (including the special case of total DFAs), such as [Car88, Epp90, Ryz19a, KM21], mostly construct a product of minimum rank iteratively, with each iteration decreasing the number of different rows or the rank of a matrix. Such an approach is inherently sequential, since the matrix in the new iteration has to depend on the previous one, which thus has to be constructed explicitly. In particular, this requires matrix multiplication at every step, which heavily increases the time complexity. In this paper, we take a different direction by strongly relying on linear algebra. While linear-algebraic arguments are used widely in the synchronising automata literature, they mostly serve to decrease the number of iterations in the iterative scheme described above. Our approach is to instead relate the rank of a zero-one matrix monoid to efficiently computable linear-algebraic properties, without explicitly constructing a matrix of minimum rank.

Our first main result is that computing the rank of a zero-one matrix monoid provided in the input by a generating set of m matrices of dimension n (or, equivalently, by a UFA with n states and m letters) is in NC^2 (5.1) and can be done in time $\mathcal{O}(mn^4)$ (6.1) and space $\mathcal{O}(n^2)$. Previously, it was not known that this problem is in NC, not even for total DFAs or finite complete codes. Moreover, the naive implementation of the polynomial time algorithm from the literature works in time $\mathcal{O}(n^{4+\omega} + mn^4)$ [Car88].

Our results rely on a new concept of weight of the matrices in a complete zero-one monoid. This theory of matrix weight, which we develop in section 4, is our main technical contribution. Matrix weight is a natural generalisation of an existing notion of weight of columns of matrices in total DFAs, which was used, e.g., in connection with the road colouring problem [Fri90, Kar01, GP16]. We show that all matrices in a zero-one matrix monoid have the same weight, and that this weight is tightly related to both the rank of the monoid and to the maximal weight of the columns and rows of its matrices (subsection 4.4). This connection allows us to reduce the computation of the monoid rank to the computation of maximal column and row weight. Then we show that we can instead compute the weight of "maximal pseudo-columns" and "maximal pseudo-rows", as they have the same weight as maximal columns and rows, respectively (subsection 4.5). Finally, we transfer linear-algebraic techniques from the literature on weighted automata to compute those weights, and thus the rank of the monoid, efficiently (section 5 and subsection 6.2).

We complement the linear-algebraic algorithms with a combinatorial algorithm, our second main contribution. While it has the same time complexity of $\mathcal{O}(mn^4)$ and a higher space complexity of $\mathcal{O}(n^3)$ in the general case (6.2), it also constructs a matrix of minimum rank in addition to computing the rank of the monoid. For total DFAs, our combinatorial algorithm runs in time $\mathcal{O}(n^3 + mn^2)$ (6.3), thus outmatching the linear-algebraic counterpart and improving upon the $\mathcal{O}(m^4n^4)$ algorithm known before [Rys92]. The key technical ingredients of our combinatorial algorithm are explained in the beginnings of subsection 6.3 and subsection 6.4. Our results on the time and space complexity of computing the rank are summarised in the table below, in the format "time complexity, space complexity, reference for both".

class	UFA	total DFA
previous best	$\mathcal{O}(n^{4+\omega}+mn^4)$, $\mathcal{O}(n^4)$ [Car88]	$\mathcal{O}(m^4n^4)$, $\mathcal{O}(n^4)$ [Rys92]
linear-algebraic	$\mathcal{O}(mn^4)$, $\mathcal{O}(n^2)$ (6.1)	$\mathcal{O}(mn^3)$, $\mathcal{O}(n^2)$ (see subsection 6.2)
combinatorial	$\mathcal{O}(mn^4)$, $\mathcal{O}(n^3)$ (6.2)	$\mathcal{O}(n^3+mn^2)$, $\mathcal{O}(n^2)$ (6.3)

3. Main definitions

Let Q be a finite set, which we view as a set of states. For $S \subseteq Q$ we write $[S]$ for the column vector $x \in \{0,1\}^Q$ such that $x(q) = 1$ if and only if $q \in S$. We may write $[q]$ for $[\{q\}]$. For a column vector $x \in \{0,1\}^Q$ we write x^T for the transpose, a row vector. For two column vectors $x_1, x_2 \in \mathbb{R}^Q$ we write $x_1 \geq x_2$ if the inequality holds component-wise. We view the elements of $\mathbb{R}^{Q\times Q}$ (and similar sets) as matrices. Vector and matrix addition and multiplication are defined in the usual way (over $\mathbb{R}$). We denote by $\langle X\rangle$ the span of a set X of vectors, i.e., the set of all linear combinations of X with real coefficients. The *real rank* of a matrix $A \in \mathbb{R}^{Q\times Q}$ is, as usual, the dimension of the column space of A over the field of the reals (which equals the dimension of the row space); i.e., $\mathsf{rank}_{\mathbb{R}}(A) = \dim\langle A[q] \mid q \in Q\rangle = \dim\langle [q]^T A \mid q \in Q\rangle$.

Let $\mathcal{A} = \{A_1, \ldots, A_m\}$ be a set of matrices from $\{0,1\}^{Q\times Q}$, and $\Sigma = \{a_1, \ldots, a_m\}$ be a finite alphabet. We associate the letters with the matrices by setting $M(a_i) = A_i$ for $1 \leq i \leq m$. Throughout this paper, when speaking about computational complexity, we assume that the input is the function $M\colon \Sigma \to \{0,1\}^{Q\times Q}$ from letters to zero-one matrices. We can extend $M\colon \Sigma \to \{0,1\}^{Q\times Q}$ naturally (and often implicitly) to $M\colon \Sigma^* \to \mathbb{Z}_{\geq 0}^{Q\times Q}$ by defining $M(a_1 \cdots a_k) = M(a_1)\cdots M(a_k)$. Thus, M is a monoid homomorphism from Σ^* to the matrix monoid $M(\Sigma^*)$ generated by $\mathcal{A} = M(\Sigma)$. Note that $M(\varepsilon) = I$, where ε denotes the empty word and I the $Q \times Q$ identity matrix. In this paper, we consider only monoid morphisms $M\colon \Sigma^* \to \mathbb{Z}_{\geq 0}^{Q\times Q}$ that are *unambiguous*, i.e., $M\colon \Sigma^* \to \{0,1\}^{Q\times Q}$. If M is unambiguous, $\mathcal{A} = M(\Sigma)$ generates a finite matrix monoid $M(\Sigma^*) \subseteq \{0,1\}^{Q\times Q}$.

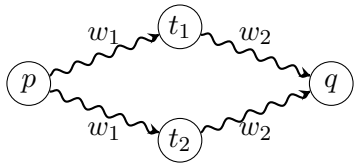


Figure 2. The configuration that is forbidden in a UFA.

Viewing the matrices as transition matrices of an automaton, we obtain a *nondeterministic finite (semi-)automaton (NFA)* (Q, Σ, Δ) with transition relation $\Delta = \{(p, a, q) \in Q \times \Sigma \times Q \mid [p]^T M(a)[q] = 1\}$. Recall that in this paper automata do not have dedicated initial or accepting states, see footnote 2 on page 2. We can extend Δ from letters to words in the usual way so that we have $\Delta = \{(p, w, q) \in Q \times \Sigma^* \times Q \mid [p]^T M(w)[q] \geq 1\}$. An NFA (Q, Σ, Δ) is *unambiguous*[3] (or *diamond-free*) if for every two states p, q and for every two words w_1, w_2 there exists at most one $t \in Q$ with $(p, w_1, t) \in \Delta$ and $(t, w_2, q) \in \Delta$; see Figure 2 for an illustration of the forbidden configuration. We denote unambiguous NFAs as UFAs. Recall from the previous section

[3]In the context of finite automata that recognise languages, the usual notion of unambiguity also depends on the choice of initial and final states, and is thus not reflected in the transition monoid. Our definition of unambiguity thus defines a strictly larger class of NFAs. Its advantage is that it is a property of the transition monoid alone, in the same way as determinism.

that deciding if an NFA is unambiguous is NL-complete. In the following, we often identify $M\colon \Sigma^* \to \{0,1\}^{Q\times Q}$ with the corresponding UFA (Q,Σ,Δ). In particular, a monoid homomorphism is unambiguous if and only if the corresponding NFA is unambiguous.

When M (or, equivalently, Δ) is clear from the context, we may write $p\cdot w = \{q\in Q \mid (p,w,q)\in\Delta\}$. Then $[p\cdot w]^T = [p]^T M(w)$. Similarly, we may write $w\cdot q = \{p\in Q \mid (p,w,q)\in\Delta\}$, so that $[w\cdot q] = M(w)[q]$. We call M *strongly connected* if for all $p,q\in Q$ there is $w\in\Sigma^*$ with $p\cdot w \ni q$. We call M *complete* if $0\notin M(\Sigma^*)$, where 0 is the zero matrix. The *real rank* of M (and of $M(\Sigma^*)$) is

$$\mathsf{rank}_{\mathbb{R}}(M) := \min\{\mathsf{rank}_{\mathbb{R}}(M(w)) \mid w\in\Sigma^*\}.$$

Note that M is complete if and only if $\mathsf{rank}_{\mathbb{R}}(M)\neq 0$.

Suppose that $|p\cdot a| = 1$ holds for every $p\in Q$ and $a\in\Sigma$, or, equivalently, that every matrix in $\mathcal{A}$ has exactly one 1 in each row. Then $|p\cdot w| = 1$ holds for every $p\in Q$ and $w\in\Sigma^*$. We call such UFAs *total deterministic finite (semi-)automata (total DFAs)* and we may write δ instead of Δ to highlight that it is a transition function $\delta\colon Q\times\Sigma\to Q$ instead of a transition relation. A total DFA (Q,Σ,δ) is complete in the sense defined above (i.e., $0\notin M(\Sigma^*)$), and for any $w\in\Sigma^*$ we have that $\mathsf{rank}_{\mathbb{R}}(M(w))$ is the number of nonzero columns in $M(w)$.

4. Main concepts and the linear algebra toolbox

In this section, we introduce the main tools that we will use for both linear-algebraic and combinatorial algorithms in later sections. Until subsection 4.6, we fix an unambiguous, complete, and strongly connected monoid morphism M. In subsection 4.6 we will show that the case where M is not strongly connected can be easily reduced to the strongly connected case.

4.1. Columns, rows and the structure of minimum rank matrices.

The concept of maximum columns and rows plays a crucial role in dealing with reachability problems in unambiguous monoid morphisms. Abusing language slightly in the following, by *column* we refer to column vectors of the form $[w\cdot q] = M(w)[q]\in\{0,1\}^Q$ where $w\in\Sigma^*$ and $q\in Q$. Similarly, a *row* is of the form $[q\cdot w]^T = [q]^T M(w)$. See Figure 3 for an example. In the case of total DFAs, all rows are of the form $[q]^T$. This fact makes total DFAs significantly simpler to deal with than general complete UFAs.

$$M(a) = \begin{pmatrix} 1&0&1&0\\ 1&0&1&0\\ 0&0&0&0\\ 0&0&0&0 \end{pmatrix} \qquad M(b) = \begin{pmatrix} 0&0&0&0\\ 0&0&0&0\\ 0&1&0&1\\ 0&1&0&1 \end{pmatrix}$$

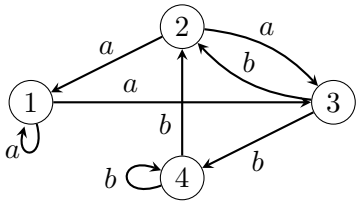


FIGURE 3. $[a\cdot 3] = M(a)[3] = [\{1,2\}]$ is a column; $[2\cdot a]^T = [2]^T M(a) = [\{1,3\}]^T$ is a row.

A column $[C]$ is called *maximal* if there is no column $[C']$ such that $[C']\neq[C]$ and $[C']\geq[C]$ (that is, $C\subset C'$). Maximal rows are defined in the same way. Recall that the inequalities are taken component-wise.

Let $A\in\{0,1\}^{m\times n}$ be a zero-one matrix. One can view $\mathsf{rank}_{\mathbb{R}}(A)$ as the least number r such that there are matrices $C\in\mathbb{R}^{m\times r}$ and $R\in\mathbb{R}^{r\times n}$ with $A = CR$. Define the *unambiguous rank* $\mathsf{rank}_{\mathsf{un}}(A)$ as the least number r such that there are matrices $C\in\{0,1\}^{m\times r}$ and $R\in\{0,1\}^{r\times n}$ such that $A = CR$. Analogously to $\mathsf{rank}_{\mathbb{R}}(M)$, define also $\mathsf{rank}_{\mathsf{un}}(M) := \min\{\mathsf{rank}_{\mathsf{un}}(M(w)) \mid w\in\Sigma^*\}$. Clearly, $\mathsf{rank}_{\mathbb{R}}(A)\leq\mathsf{rank}_{\mathsf{un}}(A)$, and the inequality can be strict, but in

4.2 below we show that $\mathsf{rank}_{\mathbb{R}}(M) = \mathsf{rank}_{\mathsf{un}}(M)$. The reason we are interested in the unambiguous rank is that 4.1 below implies that there is always a matrix with a very simple structure such that its unambiguous rank is equal to its real rank and both ranks are minimum.

In the following let us write $r := \mathsf{rank}_{\mathsf{un}}(M)$ when M is understood. A word $u \in \Sigma^*$ is *of minimum unambiguous rank* if $\mathsf{rank}_{\mathsf{un}}(M(u)) = r$. If $u \in \Sigma^*$ is of minimum unambiguous rank then so is vuw for all $v, w \in \Sigma^*$.

Words of unambiguous rank one, known as synchronising words, play an especially important role due to their applications in the theory of codes, as explained in section 2. It is easy to see that a word w has unambiguous rank one if and only if there exist $C, R \subseteq Q$ such that w maps a state p to a state q if and only if $p \in C$ and $q \in R$. For total DFAs, we moreover have that $C = Q$ and R has cardinality one.

Theorem 4.1 Césari [Cés74]**.** *Let $u \in \Sigma^*$ be of minimum unambiguous rank. There are pairwise disjoint sets $C_1, \dots, C_r \subseteq Q$ and pairwise disjoint sets $R_1, \dots, R_r \subseteq Q$ such that*

$$M(u) = \sum_{i=1}^{r} [C_i][R_i]^T.$$

Moreover, each $[C_i]$ and $[R_i]^T$ is, respectively, a maximal column and a maximal row.

This theorem will play a central role. A proof can be found in [BCKP08a, Proposition 4]. For the sake of completeness, we provide an elementary proof of the next two sections. In the case of a total DFA, R_1 is a singleton and 4.1 is fairly obvious.

In 4.1, since the C_i are pairwise disjoint and the R_i are pairwise disjoint, each $[C_i][R_i]^T$ forms, intuitively, a "combinatorial rectangle", and no such rectangle shares a row or a column with any other rectangle. The column vectors $[C_i]$ are exactly the nonzero columns of $M(u)$ and linearly independent, and the row vectors $[R_i]^T$ are exactly the nonzero rows of $M(u)$ and linearly independent. Thus, r is the number of distinct nonzero columns and also the number of distinct nonzero rows in $M(u)$. It follows that $r = \mathsf{rank}_{\mathsf{un}}(M(u)) = \mathsf{rank}_{\mathbb{R}}(M(u))$. Thus we have:

Corollary 4.2. *We have $r = \mathsf{rank}_{\mathsf{un}}(M) = \mathsf{rank}_{\mathbb{R}}(M)$.*

We can thus define $\mathsf{rank}(M)$ as $\mathsf{rank}_{\mathsf{un}}(M) = \mathsf{rank}_{\mathbb{R}}(M)$. For words $w \in \Sigma^*$ that are not of minimum unambiguous rank, we may have $\mathsf{rank}_{\mathbb{R}}(M(w)) < \mathsf{rank}_{\mathsf{un}}(M(w))$, but the rank of such matrices will rarely play a role in the following. In what follows, we call words of minimum unambiguous rank simply words of minimum rank. Since below we never refer to the real rank of words, this will not lead to any confusion.

4.2. **A proof of the first statement in Césari's theorem.** We use the following lemma.

Lemma 4.3. *Let $A \in \{0, 1, 2, \dots\}^{Q \times Q}$ be with $[Q]^T A \geq [Q]^T$ and $A[Q] \geq [Q]$. Then A is a permutation matrix* or $\sup_{i \geq 0} [Q]^T A^i [Q] = \infty$.

Proof. By a straightforward induction argument we have that $A^{i+1}[Q] \geq A^i[Q]$ holds for all $i \geq 0$. It suffices to show that A is a permutation matrix *or* for all $i \geq 0$ we have $[Q]^T A^{i+1}[Q] > [Q]^T A^i[Q]$. We suppose that A is not a permutation matrix and show by induction on i that $[Q]^T A^{i+1}[Q] > [Q]^T A^i[Q]$ holds for all $i \geq 0$.

Concerning the induction base, $i = 0$, since A is not a permutation matrix, we cannot have both $[Q]^T A = [Q]^T$ and $A[Q] = [Q]$ (in fact, we can have neither). Therefore, $[Q]^T A[Q] > [Q]^T[Q]$. For the induction step, suppose that $[Q]^T A^{i+1}[Q] > [Q]^T A^i[Q]$ holds for some $i \geq 0$. Then $A^{i+1}[Q] > A^i[Q]$; i.e., the inequality $A^{i+1}[Q] \geq A^i[Q]$ is strict in some component. Since $[Q]^T A \geq [Q]^T$, the vector $[Q]^T A$ is strictly positive in all components. It follows that $[Q]^T A A^{i+1}[Q] > [Q]^T A A^i[Q]$, as required. □

This allows us to prove the first statement of 4.1. The second statement is proved at the end of the next subsection.

Proposition 4.4. *Let $u \in \Sigma^*$ be of minimum rank. There are pairwise disjoint sets $C_1, \ldots, C_r \subseteq Q$ and pairwise disjoint sets $R_1, \ldots, R_r \subseteq Q$ such that $M(u) = \sum_{i=1}^{r} [C_i][R_i]^T$.*

Proof. Since $\mathsf{rank_{un}}(M(u)) = r$, there are $C \in \{0,1\}^{Q \times r}$ and $R \in \{0,1\}^{r \times Q}$ with $M(u) = CR$. For notational convenience, take a finite set X with $|X| = r$ and write $C \in \{0,1\}^{Q \times X}$ and $R \in \{0,1\}^{X \times Q}$. Let $w \in \Sigma^*$. Define $P(w) := RM(w)C \in \{0,1,2,\ldots\}^{X \times X}$. For all $i \geq 0$ we have $CP(w)^i R = (CRM(w))^i CR = M((uw)^i u) \in \{0,1\}^{Q \times Q}$. Since C has no zero columns and R has no zero rows, we have $P(w)^i \in \{0,1\}^{X \times X}$ for all i. Further, since $\mathsf{rank_{un}}(P(w)) \geq \mathsf{rank_{un}}(CP(w)R) = \mathsf{rank_{un}}(M(uwu)) = r$, we have $\mathsf{rank_{un}}(P(w)) = r$. In particular, $P(w)$ does not have a zero row or column; i.e., $P(w)[X] \geq [X]$ and $[X]^T P(w) \geq [X]^T$. It follows from 4.3 that $P(w)$ is a permutation matrix. As $w \in \Sigma^*$ was arbitrary, $P(w)$ is a permutation matrix for all w.

Let $q_1 \in Q$, and let $q_2 \in Q$ and $y \in X$ be such that $[q_2]^T C[y] = 1$. By strong connectedness, there is $w \in \Sigma^*$ with $[q_1]^T M(w)[q_2] = 1$. Thus,

$$P(w)[y] \;=\; RM(w)C[y] \;\geq\; R[q_1][q_1]^T M(w)[q_2][q_2]^T C[y] \;=\; R[q_1]\,.$$

As $P(w)$ is a permutation matrix and $q_1 \in Q$ was arbitrary, it follows that all nonzero columns of R are of the form $[x]$ for $x \in X$. Similarly, all nonzero rows of C are of the form $[x]^T$ for $x \in X$.

For each $x \in X$, define $C_x, R_x \subseteq Q$ such that $[C_x] = C[x]$ and $[R_x]^T = [x]^T R$. The R_x are pairwise disjoint, as if there was $q \in R_x \cap R_y$ with $x \neq y$, then $R[q] \geq [\{x, y\}]$, contradicting what we proved in the previous paragraph. Similarly, the C_x are pairwise disjoint. Finally, we have

$$M(u) \;=\; CR \;=\; \sum_{x \in X} C[x][x]^T R \;=\; \sum_{x \in X} [C_x][R_x]^T\,,$$

as desired. □

4.3. **The weight of columns and rows.** The results in this subsection, about the column and row vectors that appear in the matrices $M(w)$, are mostly due to [Cés74]; see also [BCKP08a, Section 3]. Since a notion of column and row weight will be crucial for us in the later development, we phrase and prove the results around these concepts, but we do not view the lemmas of this subsection as novel.

Define $\overline{A} = \frac{1}{|\Sigma|} \sum_{a \in \Sigma} M(a) \in [0,1]^{Q \times Q}$. Since M is strongly connected, $\overline{A}$ is irreducible. Since M is unambiguous, the spectral radius of $\overline{A}$ is at most 1, and since M is complete, it is at least 1. Thus, the spectral radius of $\overline{A}$ equals 1. Since $\overline{A}$ is irreducible, it follows from basic Perron-Frobenius theory that $\overline{A}$ has an eigenvalue 1 and every right eigenvector with eigenvalue 1 is a multiple of a strictly positive vector, say $\beta \in \mathbb{R}^Q_{>0}$. Since $\overline{A}$ has only rational entries, we can assume $\beta \in \mathbb{Q}^Q_{>0}$. Similarly for left eigenvectors. Therefore, there are $\alpha, \beta \in \mathbb{Q}^Q_{>0}$ with $\alpha^T \overline{A} = \alpha^T$ and $\overline{A}\beta = \beta$. Without loss of generality, we assume that $\alpha^T \beta = 1$.

In the total DFA case, since $M(a)[Q] = [Q]$ for all $a \in \Sigma$, we have $\overline{A}[Q] = [Q]$ and so it is natural to take $\beta = [Q]$. In that case, $\alpha^T[Q] = \alpha^T \beta = 1$ means that $\alpha^T = \alpha^T \overline{A}$ is the (unique) stationary distribution of the Markov chain whose transition probabilities are given by the row-stochastic matrix $\overline{A}$; intuitively, in this Markov chain a letter $a \in \Sigma$ is picked uniformly at random in every step.

Define the *weight* of a column y and of a row x^T by $\alpha^T y \in \mathbb{R}$ and $x^T \beta \in \mathbb{R}$, respectively. Denote the maximum column weight and the maximum row weight by mcw and mrw, respectively, i.e.,

$$\mathsf{mcw} \;:=\; \max\{\alpha^T y \mid y \text{ is a column}\} \quad \text{and} \quad \mathsf{mrw} \;:=\; \max\{x^T \beta \mid x^T \text{ is a row}\}\,.$$

A column y is called *of maximum weight* if $\alpha^T y = \mathsf{mcw}$, and analogously for rows. In the total DFA case, every row is of the form $[q]^T$ for some $q \in Q$, hence every row is of maximum weight.

For $q \in Q$ define

$$\mathsf{Mer}(q) \;:=\; \{q' \in Q \mid \exists\, S \supseteq \{q, q'\} \text{ such that } [S] \text{ is a column}\}\,.$$

Intuitively, $\mathsf{Mer}(q)$ consists of the states that can "appear" in a column together with q, or, equivalently, the states that are "mergeable" with q (that is, can be mapped to the same state by a word). Note that $q \in \mathsf{Mer}(q)$.

The following lemmas apply symmetrically also to rows.

Lemma 4.5. *Let $v \in \Sigma^*$ and $q \in Q$ be such that $[v \cdot q]$ is a column of maximum weight. Then we have:*

(a) $[uv \cdot q]$ is a column of maximum weight for all $u \in \Sigma^*$;
(b) $v \cdot q' = \emptyset$ holds for all $q' \in \mathsf{Mer}(q) \setminus \{q\}$.

Proof. Towards (a), let $a \in \Sigma$. It suffices to prove that $M(a)[v \cdot q]$ is of maximum weight. We have $\alpha^T[v \cdot q] = \alpha^T \overline{M}[v \cdot q] = \frac{1}{|\Sigma|} \sum_{b \in \Sigma} \alpha^T M(b)[v \cdot q]$. Thus, if $\alpha^T M(a)[v \cdot q] < \alpha^T[v \cdot q]$, then there would also exist $b \in \Sigma$ with $\alpha^T M(b)[v \cdot q] > \alpha^T[v \cdot q]$, contradicting that $[v \cdot q]$ is of maximum weight. So $M(a)[v \cdot q]$ is of maximum weight.

Towards (b), let $q' \in \mathsf{Mer}(q) \setminus \{q\}$. Since M is strongly connected, there is $w \in \Sigma^*$ with $[w \cdot q] \geq [q] + [q']$. Thus, $[vw \cdot q] \geq [v \cdot q] + [v \cdot q']$. It follows that, since $[v \cdot q]$ is of maximum weight, so is $[vw \cdot q]$. Hence, $[v \cdot q'] = 0$. □

Lemma 4.6. *Let $S \subseteq Q$ be such that $[S]$ is a column which is not of maximum weight.*

(a) There is $S' \supsetneq S$ such that $[S']$ is a column of maximum weight.
(b) There is $u \in \Sigma^*$ with $M(u)[S] = 0$.

Proof. Let $v \in \Sigma^*$ and $q \in Q$ be such that $[v \cdot q]$ is not of maximum weight.

Towards (a), let $w \in \Sigma^*$ and $t \in Q$ be such that $[w \cdot t]$ is of maximum weight and $w \cdot t \ni q$. By 4.5 (a), $[vw \cdot t]$ is of maximum weight. Since $vw \cdot t \supseteq v \cdot q$ and $[v \cdot q]$ is not of maximum weight, we have $vw \cdot t \supsetneq v \cdot q$.

Towards (b), let $p \in S' \setminus S$. We have $S \subseteq S' \subseteq \mathsf{Mer}(p)$ and thus $S \subseteq \mathsf{Mer}(p) \setminus \{p\}$. Let $u \in \Sigma^*$ be such that $[u \cdot p]$ is of maximum weight. Then it follows from 4.5 (b) that $M(u)[S] = 0$. □

Lemma 4.7. *A column (respectively, row) is maximal if and only if it is of maximum weight.*

Proof. If a column is of maximum weight, it is clearly maximal. Conversely, if a column is not of maximum weight, then 4.6 (a) says it is not maximal. □

An important property that we will need later is that the set of maximal columns is closed under left multiplication by matrices from the monoid, as stated in the following lemma. Note that this is no longer true without the completeness assumption, and is the key reason why the case of complete matrix monoids is easier to deal with.

Lemma 4.8. *Let $v \in \Sigma^*$ and $q \in Q$ be such that $[v \cdot q]$ is a maximal column. Then $[uv \cdot q]$ is a maximal column for all $u \in \Sigma^*$.*

Proof. Immediate from 4.5 (a) and 4.7. □

The following lemma will be useful later to construct minimum rank matrices from maximal columns and rows.

Lemma 4.9. *Let $w \in \Sigma^*$ be such that all nonzero columns and rows in $M(w)$ are maximal. Then ww is of minimum rank.*

Proof. Define $S := \{q \in Q \mid w \cdot q \neq \emptyset \neq q \cdot w\}$. Since

$$M(ww) \;=\; M(w)IM(w) \;=\; M(w)\big(\sum_{q\in Q}[q][q]^T\big)M(w) \;=\; \sum_{q\in Q}[w\cdot q][q\cdot w]^T \;=\; \sum_{q\in S}[w\cdot q][q\cdot w]^T\,,$$

we have $\mathsf{rank}_{\mathsf{un}} M(ww) \leq |S|$.
Let $u \in \Sigma^*$ be of minimum rank. Suppose there were a state $p \in Q$, two states $q, q' \in S$ and uw-labelled paths from p to q and also from p to q'. Then $p \cdot uww \supseteq q \cdot w \cup q' \cdot w$, contradicting the maximality of the row $[q \cdot w]^T$. Therefore, the sets $uw \cdot q$, where $q \in S$, are pairwise disjoint. Since the columns $[w \cdot q]$ for $q \in S$ are maximal, by 4.8 the columns $[uw \cdot q]$ are also maximal and in particular nonzero. Moreover, these columns appear in $M(uww)$, as $q \cdot w \neq \emptyset$ for all $q \in S$. Therefore, $|S| \leq \mathsf{rank}_{\mathsf{un}} M(uww)$. Since u is of minimum rank, we have $\mathsf{rank}_{\mathsf{un}} M(uww) \leq r$. By combining all inequalities we obtain that $\mathsf{rank}_{\mathsf{un}} M(ww) \leq r$. Hence, ww is of minimum rank. □

We can now complete the proof of 4.1.

Proof of 4.1. The first statement is 4.4. We prove the second statement only for columns, as the proof for rows is analogous. Towards a contradiction, suppose that $[C_i]$ is a non-maximal column; without loss of generality, say $i = r$. By 4.7, $[C_i]$ is not of maximum weight. By 4.6 (b), there is $v \in \Sigma^*$ such that $M(v)[C_r] = 0$. Then

$$M(vu) \;=\; \sum_{i=1}^{r} M(v)[C_i][R_i]^T \;=\; \sum_{i=1}^{r-1} M(v)[C_i][R_i]^T \;=\; CR^T\,,$$

where $C, R \in \{0,1\}^{Q\times(r-1)}$ are the matrices whose ith columns are $M(v)[C_i]$ and $[R_i]$, respectively. Hence, $\mathsf{rank}_{\mathsf{un}}(M(vu)) \leq r-1$, contradicting the definition of r. □

4.4. **Weight preservation property and minimum rank.** Every word w of minimum rank in a total DFA induces a partition of the state set into subsets of states mapped by w to the same state (that is, into columns). It was observed by Friedman in [Fri90] that all sets of such a partition have the same weight. This observation has many applications to variations of the road colouring problem [Fri90, Kar01, GP16, KRV19]. Moreover, it was proved in [GP16, Theorem 6], again in connection with road colouring, that for every w the weights of all columns in $M(w)$ sum up to 1 (assuming $\beta = [Q]$ as suggested previously). This can be seen as a weight preservation property: the total weight of columns in the matrix of a word is preserved under multiplication by any matrix from the monoid. As a result we get that $1 = r \cdot \mathsf{mcw}$, and hence $r = \frac{1}{\mathsf{mcw}}$. The proof of the weight preservation property for total DFAs is quite simple and relies on the fact that for a state q and a word w the set $q \cdot w$ is always a singleton. For complete UFAs this is no longer true; in particular, $q \cdot w$ can be the empty set, thus permanently "losing" some weight collected in q. Hence, a more refined property is required. The following result provides such a property. It also turns out that its proof requires more sophisticated techniques than in the total DFA case.

Theorem 4.10. *For all $w \in \Sigma^*$ we have $1 = \alpha^T M(w)\beta = r \cdot \mathsf{mcw} \cdot \mathsf{mrw}$.*

Similarly to the total DFA case, this result allows us to reduce computing r to computing mcw and mrw, which we will use later in our algorithms. Recall that we have defined α^T and β so that $\alpha^T\beta = 1$.

Towards a proof of 4.10 we first prove the following lemma.

Lemma 4.11. *Let $u \in \Sigma^*$ be of minimum rank. Then $\alpha^T M(u)\beta = r \cdot \mathsf{mcw} \cdot \mathsf{mrw}$.*

Proof. Let $M(u) = \sum_{i=1}^{r} [C_i][R_i]^T$ be as in 4.1. Each $[C_i]$ and each $[R_i]$ is of maximum weight. Thus,

$$\alpha^T M(u)\beta \;=\; \sum_{i=1}^{r} \alpha^T [C_i][R_i]^T \beta \;=\; \sum_{i=1}^{r} \mathsf{mcw} \cdot \mathsf{mrw} \;=\; r \cdot \mathsf{mcw} \cdot \mathsf{mrw}\,. \qquad \square$$

We also need the following proposition.

Proposition 4.12. *Let $x \in \mathbb{R}^Q$ and $c \in \mathbb{R}$ be such that $x^T M(u)\beta = c$ holds for all $u \in \Sigma^*$ of minimum rank. Then $x^T M(w)\beta = c$ holds for all $w \in \Sigma^*$.*

Proof. Let $w \in \Sigma^*$. Let $u \in \Sigma^*$ be of minimum rank. Recall that every word that contains u as a factor is of minimum rank. For every $k \geq 0$, partition Σ^k into sets $W_0(k)$ and $W_1(k)$ so that $W_0(k) = \Sigma^k \cap (\Sigma^* u \Sigma^*)$ and $W_1(k) = \Sigma^k \setminus (\Sigma^* u \Sigma^*)$; i.e., $W_0(k), W_1(k)$ are the sets of length-k words that do or do not contain u as a factor, respectively. For all $v \in W_0(k)$ both v and wv are of minimum rank. Thus, we have $x^T M(wv)\beta = c$ for all $v \in W_0(k)$. It follows that

$$\sum_{v \in W_0(k)} \frac{x^T M(wv)\beta}{|W_0(k)|} \;=\; c \quad \text{for all } k \geq 0. \tag{4.1}$$

Let $d > 0$ be such that $|x^T A \beta| \leq d$ for all $A \in \{0,1\}^{Q \times Q}$. Then we have

$$\sum_{v \in W_1(k)} \frac{|x^T M(wv)\beta|}{|W_1(k)|} \;\leq\; d \quad \text{for all } k \geq 0. \tag{4.2}$$

Let $m \geq 0$. Define $p_1(m) := \frac{|W_1(m|u|)|}{|\Sigma|^{m|u|}}$. We can view $p_1(m)$ as the probability of picking a word in $W_1(m|u|)$ when a word of length $m|u|$ is picked uniformly at random. We have $p_1(m) \leq \left(1 - \frac{1}{|\Sigma|^{|u|}}\right)^m$, as in order to avoid u as a factor, it has to be avoided in each of the m consecutive blocks of length $|u|$. Thus, $\lim_{m\to\infty} p_1(m) = 0$. We have

$$\begin{aligned}
x^T M(w)\beta \;&=\; x^T M(w)\overline{A}\beta \;=\; x^T M(w)\overline{A}^{m|u|}\beta \;=\; \frac{1}{|\Sigma|^{m|u|}} \sum_{v \in \Sigma^{m|u|}} x^T M(wv)\beta \\
&=\; \frac{|W_0(m|u|)|}{|\Sigma|^{m|u|}} \sum_{v \in W_0(m|u|)} \frac{x^T M(wv)\beta}{|W_0(m|u|)|} + \frac{|W_1(m|u|)|}{|\Sigma|^{m|u|}} \sum_{v \in W_1(m|u|)} \frac{x^T M(wv)\beta}{|W_1(m|u|)|} \\
&=\; (1 - p_1(m)) \cdot c + p_1(m) \cdot \sum_{v \in W_1(m|u|)} \frac{x^T M(wv)\beta}{|W_1(m|u|)|} \qquad \text{(by Equation 4.1).}
\end{aligned}$$

With Equation 4.2 it follows that

$$|x^T M(w)\beta - c| \;\leq\; p_1(m)(|c| + d)\,.$$

Since this holds for all $m \geq 0$ and $\lim_{m\to\infty} p_1(m) = 0$, we conclude that $x^T M(w)\beta = c$. $\square$

Now we prove 4.10.

Proof of 4.10. It follows from 4.11 and 4.12 that

$$\alpha^T M(w)\beta \;=\; r \cdot \mathsf{mcw} \cdot \mathsf{mrw} \qquad \text{for all } w \in \Sigma^*.$$

With $w = \varepsilon$ we obtain $1 = \alpha^T\beta = \alpha^T M(\varepsilon)\beta = r \cdot \mathsf{mcw} \cdot \mathsf{mrw}$, as required. □

4.5. **Maximal pseudo-columns.** In this subsection, we define maximal pseudo-columns, which are vectors that can be seen as a relaxation of the notion of maximal columns. We show that the weight of a maximal pseudo-column is equal to the weight of a maximal column, and a maximal pseudo-column is a solution of a system of linear equations, and thus can be computed efficiently. By invoking 4.10, this will allow us to efficiently compute r.

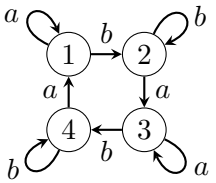
Denote by $\mathsf{MCol} \subseteq \{0,1\}^Q$ the set of maximal columns. By 4.1 (bearing in mind also 4.2), the vector space spanned by all maximum columns, $\langle \mathsf{MCol} \rangle$, is at least r-dimensional:

Proposition 4.13. *We have* $r \le \dim \langle \mathsf{MCol} \rangle$.

One might hypothesise that $r = \dim \langle \mathsf{MCol} \rangle$ or even that all minimum-rank matrices have the same r nonzero (hence, maximum) columns. The following example shows that neither is the case in general, not even for total DFAs.

Example 4.14. Consider the total DFA with $\Sigma = \{a, b\}$ and

$$M(a) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix} \qquad M(b) = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



By symmetry, we have $\alpha^T = \begin{pmatrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \end{pmatrix}$. Since no word maps states 1 and 3 to the same state, we have $r = 2$; i.e., a and b are both minimum rank. Further, MCol consists exactly of the four nonzero columns in $M(a)$ and $M(b)$. Their span $\langle \mathsf{MCol} \rangle$ is 3-dimensional, as $\begin{pmatrix} 1 & -1 & 1 & -1 \end{pmatrix}$ is orthogonal to each maximum column. Thus, $r = 2 < 3 = \dim \langle \mathsf{MCol} \rangle < 4 = |\mathsf{MCol}|$.

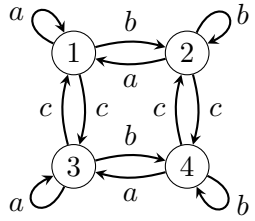
Define the vector space $U := \langle \alpha^T M(w) - \alpha^T \mid w \in \Sigma^* \rangle$. Intuitively, it is the set of all differences of weight distributions over the states before and after a word is applied. Notice that for all $w_1, w_2 \in \Sigma^*$ we have $\alpha^T M(w_1) - \alpha^T M(w_2) \in U$. Later (see the proof of 5.2 below) we show that U is closed under post-multiplication with $M(a)$ for all $a \in \Sigma$. Such "forward spaces" play an important role in weighted automata; see, e.g., [Kie20]. Denote the orthogonal complement of U by $U^\perp$; i.e., $U^\perp = \{y \in \mathbb{R}^Q \mid \forall\, w \in \Sigma^* :\ \alpha^T M(w) y = \alpha^T y\}$. Intuitively, it is the set of vectors whose weight does not change under pre-multiplication with $M(w)$ for any w (where by the weight of a vector y we understand $\alpha^T y$). Clearly, $\dim U + \dim U^\perp = |Q|$. The following proposition follows immediately from 4.8.

Proposition 4.15. *We have* $\mathsf{MCol} \subseteq U^\perp$.

It follows that $\langle \mathsf{MCol} \rangle$ is a subspace of $U^\perp$. With 4.13, we have $r \le \dim \langle \mathsf{MCol} \rangle \le \dim U^\perp$. One might hypothesise that $\langle \mathsf{MCol} \rangle = U^\perp$. The following example shows that this is not the case in general, not even for total DFAs.

Example 4.16. Consider the DFA with $\Sigma = \{a, b, c\}$ and

$$M(a) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \; M(b) = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \; M(c) = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}.$$



We have $\mathsf{Mer}(1) = \mathsf{Mer}(2) = \{1, 2\}$ and $\mathsf{Mer}(3) = \mathsf{Mer}(4) = \{3, 4\}$. Thus, $\mathsf{MCol} = \{\begin{pmatrix}1 & 1 & 0 & 0\end{pmatrix}^T, \begin{pmatrix}0 & 0 & 1 & 1\end{pmatrix}^T\}$. Hence, $\dim \langle \mathsf{MCol} \rangle = 2$.

On the other hand, by symmetry we have $\alpha^T = \begin{pmatrix}\frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4}\end{pmatrix}$. For any $w \in \Sigma^*$,

$$\alpha^T M(w) \; = \; \begin{cases} \begin{pmatrix}\frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4}\end{pmatrix} & \text{if } w \in \{c\}^* \\ \begin{pmatrix}\frac{1}{2} & 0 & \frac{1}{2} & 0\end{pmatrix} & \text{if the last non-}c\text{ letter in } w \text{ is } a \\ \begin{pmatrix}0 & \frac{1}{2} & 0 & \frac{1}{2}\end{pmatrix} & \text{if the last non-}c\text{ letter in } w \text{ is } b\,. \end{cases}$$

It follows that $U = \langle \begin{pmatrix}1 & -1 & 1 & -1\end{pmatrix} \rangle$. Thus, $\dim U^\perp = 4 - 1 = 3 > 2 = \dim \langle \mathsf{MCol} \rangle$, and $\langle \mathsf{MCol} \rangle$ is a strict subspace of $U^\perp$. For example, the vector $\begin{pmatrix}1 & 0 & 0 & 1\end{pmatrix}^T$ is in $U^\perp$ but not in $\langle \mathsf{MCol} \rangle$.

Although the dimension of $U^\perp$ does not generally equal r, the vector space $U^\perp$ turns out useful for computing r. Recall that, by 4.10, we can obtain r by computing mcw (and, symmetrically, mrw). Recall that we define for $q \in Q$

$$\mathsf{Mer}(q) \; := \; \{q' \in Q \mid \exists\, S \supseteq \{q, q'\} \text{ such that } [S] \text{ is a column}\}\,.$$

We will need the following lemma which is easy to prove.

Lemma 4.17. *Let $v \in \Sigma^*$ and $q \in Q$ be such that $[v \cdot q]$ is a maximal column. Then $v \cdot q' = \emptyset$ holds for all $q' \in \mathsf{Mer}(q) \setminus \{q\}$.*

We call a vector $y \in U^\perp$ a *maximal pseudo-column* if there is $q \in Q$ with $y(q) = 1$ and $y(q') = 0$ for all $q' \notin \mathsf{Mer}(q)$. This notion, which is closely related to the "pseudo-cuts" from [KW19b], can be seen as a relaxation of the notion of a maximal column: clearly, every maximal column is a maximal pseudo-column, but the converse is not true, since a maximal pseudo-column is not necessarily a vector over $\{0, 1\}$, let alone a *column* in the strict sense, i.e., of the form $[w \cdot p]$. The following lemma however shows that the weight of a maximal pseudo-column is equal to the weight of a maximal column. We will later show that computing the former can be done in NC^2.

Lemma 4.18. *Let y be a maximal pseudo-column. Then $\alpha^T y = \mathsf{mcw}$.*

Proof. Let $q \in Q$ be such that $y(q) = 1$ and $y(q') = 0$ for all $q' \notin \mathsf{Mer}(q)$. Let $w \in \Sigma^*$ be such that $[w \cdot q]$ is a maximal column. We have

$$\begin{aligned}
\alpha^T y &= \alpha^T M(w) y && (y \in U^\perp)\\
&= \sum_{q' \in Q} y(q') \alpha^T [w \cdot q'] &&\\
&= \sum_{q' \in \mathsf{Mer}(q)} y(q') \alpha^T [w \cdot q'] && (y(q') = 0 \text{ for } q' \notin \mathsf{Mer}(q))\\
&= \alpha^T [w \cdot q] + \sum_{q' \in \mathsf{Mer}(q) \setminus \{q\}} y(q') \alpha^T [w \cdot q'] && (y(q) = 1)\\
&= \alpha^T [w \cdot q] && (4.17)\\
&= \mathsf{mcw} && (\text{by the choice of } w, q).
\end{aligned}$$

□

Example 4.19. We continue 4.14. We have $U = \langle \begin{pmatrix} 1 & -1 & 1 & -1 \end{pmatrix} \rangle$ and $\mathsf{Mer}(2) = \{1, 2, 3\}$. Let $y = \begin{pmatrix} 4/3 & 1 & -1/3 & 0 \end{pmatrix}^T$. Then $y \in U^\perp$. Since $y(2) = 1$ and $y(4) = 0$, vector y is a maximal pseudo-column. Thus, by 4.18, $\mathsf{mcw} = \alpha^T y = \begin{pmatrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \end{pmatrix} y = \frac{1}{2}$.

Theorem 4.20. *Let Γ be a basis of U, and let $q \in Q$. Then the following linear system for $y \in \mathbb{R}^Q$ has a solution, and all its solutions are maximal pseudo-columns:*

$$\begin{aligned}
\gamma^T y &= 0 \quad \text{for all } \gamma^T \in \Gamma\\
y(q) &= 1\\
y(q') &= 0 \quad \text{for all } q' \notin \mathsf{Mer}(q)\,.
\end{aligned}$$

Proof. By 4.15, any maximal column solves the linear system. Let $y \in \mathbb{R}^Q$ be a solution of the linear system. The equations on the first line guarantee that $y \in U^\perp$. Then, the equations on the second and third line guarantee that y is a maximal pseudo-column. □

4.6. **Dealing with the non-strongly connected case.** The following lemma shows that in order to compute the minimum rank we can focus on the strongly connected case.

Proposition 4.21. *Let $M : \Sigma \to \{0,1\}^{Q \times Q}$ be an unambiguous matrix monoid morphism. Suppose that $Q_1 \cup Q_2 = Q$ is a partition of Q such that for all $w \in \Sigma^*$ it holds that $[Q_2]^T M(w) [Q_1] = 0$; i.e., for all $w \in \Sigma^*$ matrix $M(w)$ has the block form $M(w) = \begin{pmatrix} M_1(w) & M_{12}(w) \\ 0 & M_2(w) \end{pmatrix}$, where $M_1(w) \in \{0,1\}^{Q_1 \times Q_1}$ and $M_{12}(w) \in \{0,1\}^{Q_1 \times Q_2}$ and $M_2(w) \in \{0,1\}^{Q_2 \times Q_2}$. We have $\mathsf{rank}_{\mathbb{R}}(M) = \mathsf{rank}_{\mathbb{R}}(M_1) + \mathsf{rank}_{\mathbb{R}}(M_2)$ and $\mathsf{rank}_{\mathsf{un}}(M) = \mathsf{rank}_{\mathsf{un}}(M_1) + \mathsf{rank}_{\mathsf{un}}(M_2)$.*

Proof. It is a well-known property of upper-triangular block matrices that $\mathsf{rank}(M(w)) \geq \mathsf{rank}(M_1(w)) + \mathsf{rank}(M_2(w))$; see, e.g., [RAH13, Chapter 0.9.4]. Concerning the analogous property of $\mathsf{rank}_{\mathsf{un}}$, let $M(w) = CR$ for some 0/1 matrices C, R. Then R has at least $\mathsf{rank}_{\mathsf{un}}(M_2(w))$ rows whose Q_1-components are all 0, and at least $\mathsf{rank}_{\mathsf{un}}(M_1(w))$ rows whose Q_1-components are not all 0. It follows that R has at least $\mathsf{rank}_{\mathsf{un}}(M_1(w)) + \mathsf{rank}_{\mathsf{un}}(M_2(w))$ rows. Since the factorization $M(w) = CR$ was arbitrary, we conclude that

$$\mathsf{rank}_{\mathsf{un}}(M(w)) \geq \mathsf{rank}_{\mathsf{un}}(M_1(w)) + \mathsf{rank}_{\mathsf{un}}(M_2(w)).$$

Now let $\mathsf{rk} \in \{\mathsf{rank}, \mathsf{rank_{un}}\}$ and $w_1, w_2 \in \Sigma^*$ be such that $\mathsf{rk}(M_1(w_1)) = r_1$ and $\mathsf{rk}(M_2(w_2)) = r_2$. With what we have shown in the first paragraph, it suffices to show that $\mathsf{rk}(M(w_1w_2)) \leq r_1 + r_2$. For some matrices A_1, A_2, B_1, B_2 we have

$$\begin{aligned} M(w_1w_2) \;=\; M(w_1)M(w_2) \;&=\; \begin{pmatrix} M_1(w_1) & A_1 \\ 0 & A_2 \end{pmatrix} \begin{pmatrix} B_1 & B_2 \\ 0 & M_2(w_2) \end{pmatrix} \\ &=\; \begin{pmatrix} M_1(w_1) \\ 0 \end{pmatrix} \begin{pmatrix} B_1 & B_2 \end{pmatrix} + \begin{pmatrix} A_1 \\ A_2 \end{pmatrix} \begin{pmatrix} 0 & M_2(w_2) \end{pmatrix} . \end{aligned}$$

The first summand has rank at most $\mathsf{rk} \begin{pmatrix} M_1(w_1) \\ 0 \end{pmatrix} = r_1$, and similarly the second summand has rank at most r_2. So we can write both ($i = 1, 2$) summands as C_iR_i, where C_i has at most r_i columns. Hence,

$$M(w_1w_2) \;=\; C_1R_1 + C_2R_2 \;=\; \begin{pmatrix} C_1 & C_2 \end{pmatrix} \begin{pmatrix} R_1 \\ R_2 \end{pmatrix} .$$

It follows that $\mathsf{rk}(M(w_1w_2)) \leq r_1 + r_2$. □

By a straightforward induction it follows from 4.21 that the minimum rank of an unambiguous matrix monoid is the sum of the minimum ranks of its strongly connected components (where "incomplete" components count as having rank 0).

5. Computing the rank in NC^2

In this section, we prove our first main result, which is as follows.

Theorem 5.1. *The problem of computing the (real) rank of an unambiguous matrix monoid is in* NC^2.

In order to use 4.20, we need the following lemma. We use the notation defined in the previous section. Recall that we defined $U := \langle \alpha^T M(w) - \alpha^T \mid w \in \Sigma^* \rangle$.

Lemma 5.2. *If M is strongly connected, one can compute a basis of U in* NC^2.

For each $a \in \Sigma$ define $M'(a) := M(a) - I \in \{-1, 0, 1\}^{Q \times Q}$ and extend M' to $M' : \Sigma^* \to \mathbb{Z}^{Q \times Q}$ by defining $M'(a_1 \cdots a_k) = M'(a_1) \cdots M'(a_k)$. Define

$$U' := \langle \alpha^T M'(w) \mid w \in \Sigma^+ \rangle .$$

Note that here w ranges over Σ^+, i.e., nonempty words, only. By definition, U' is closed under right multiplication by $M'(a)$ for all $a \in \Sigma$. We first show the following lemma.

Lemma 5.3. *We have $U = U'$.*

Proof. For the inclusion $U \subseteq U'$, we prove by induction on $i \geq 0$ that for all length-i words $w \in \Sigma^i$ we have $\alpha^T(M(w) - I) \in U'$. Concerning the induction base, $i = 0$, we have $\alpha^T(M(\varepsilon) - I) = 0 \in U'$. Concerning the induction step, let $i \geq 0$, and let $w \in \Sigma^i$ and $a \in \Sigma$. We have

$$\begin{aligned} \alpha^T(M(wa) - I) \;&=\; \alpha^T(M(w) - I)M(a) + \alpha^T(M(a) - I) \\ &=\; \alpha^T(M(w) - I)(M(a) - I) + \alpha^T(M(w) - I) + \alpha^T(M(a) - I) \\ &=\; \alpha^T(M(w) - I)M'(a) + \alpha^T(M(w) - I) + \alpha^T M'(a) . \end{aligned}$$

It holds that $\alpha^T M'(a) \in U'$, and, by the induction hypothesis, $\alpha^T(M(w) - I) \in U'$. It follows that $\alpha^T(M(wa) - I) \in U'$.

For the converse, $U' \subseteq U$, we proceed similarly by induction. Concerning the induction base, $i = 1$, for all $a \in \Sigma$ we have $\alpha^T M'(a) = \alpha^T(M(a) - I) \in U$. Concerning the induction step, let $i \geq 1$, and let $w \in \Sigma^i$ and $a \in \Sigma$. By the induction hypothesis there are $n \leq |Q|$ and $w_1, \ldots, w_n \in \Sigma^*$ and $\lambda_1, \ldots, \lambda_n \in \mathbb{R}$ such that $\alpha^T M'(w) = \sum_{i=1}^n \lambda_i \alpha^T(M(w_i) - I)$. Thus, we have

$$\begin{aligned} \alpha^T M'(wa) &= \alpha^T M'(w)(M(a) - I) = \sum_{i=1}^n \lambda_i \alpha^T (M(w_i) - I)(M(a) - I) \\ &= \sum_{i=1}^n \lambda_i \alpha^T \big((M(w_i a) - I) - (M(a) - I) - (M(w_i) - I)\big) \in U\,, \end{aligned}$$

as required. □

Proof of 5.2. For each $a \in \Sigma$ define $U'_a := \langle \alpha^T M'(a) M'(w) \mid w \in \Sigma^* \rangle$. Using the technique from [KMW17, Section 4.2] (see [Kie20, Proposition 5.2] for a clearer explanation), for each $a \in \Sigma$ one can compute[4] a basis of U'_a in NC^2. The union of these bases, say $\Gamma = \{\gamma_1^T, \ldots, \gamma_n^T\}$ for some $n \leq |\Sigma||Q|$, spans U', which equals U by 5.3. To shrink Γ to a basis of U, for each $i \in \{1, \ldots, n\}$ include γ_i^T in the basis if and only if $\dim \langle \gamma_1^T, \ldots, \gamma_{i-1}^T \rangle < \dim \langle \gamma_1^T, \ldots, \gamma_i^T \rangle$. The latter (rank) computation can be done in NC^2 [IMR80]. □

Now we can prove 5.1.

Proof of 5.1. Let $M : \Sigma \to \{0,1\}^{Q \times Q}$ be an unambiguous monoid morphism. Its strongly connected components can be computed in $\mathsf{NL} \subseteq \mathsf{NC}^2$. It follows from the proof of [KM21, Proposition 3] that one can check each component for completeness in NC^2, since a zero-one monoid contains the zero matrix if and only if the joint spectral radius of the set of its generators is strictly less than one [KM21]. Therefore, using 4.21, we can assume in the rest of the proof that M is complete and strongly connected.

We use the fact that one can compute a solution of a possibly singular linear system of equations in NC^2 [BvzGH82, Section 5]. First, compute in NC^2 vectors $\alpha, \beta \in \mathbb{Q}_{>0}^Q$ with $\alpha^T \overline{A} = \alpha^T$ and $\overline{A}\beta = \beta$ and $\alpha^T \beta = 1$. Using 5.2 compute in NC^2 a basis of U. Choose an arbitrary $q \in Q$ and compute $\mathsf{Mer}(q)$ in $\mathsf{NL} \subseteq \mathsf{NC}^2$ with a reachability analysis. Then, solve the linear system from 4.20 to compute in NC^2 a maximal pseudo-column $y \in \mathbb{Q}^Q$. Hence, using 4.18, we can compute $\mathsf{mcw} = \alpha^T y$ in NC^2. Symmetrically, we can compute mrw in NC^2. Finally, by 4.10, we can compute $r = \frac{1}{\mathsf{mcw} \cdot \mathsf{mrw}}$ in NC^2. □

Example 5.4. We continue 4.14 and 4.19. Since M is a total DFA, it is natural to take $\beta = [Q]$. Note that $\alpha^T \beta = 1$. Since every row is of the form $[q]^T$ for some q, we have $\mathsf{mrw} = 1$. Recall from 4.19 that $\mathsf{mcw} = \frac{1}{2}$. With 4.10 we conclude that $r = \frac{1}{\mathsf{mcw} \cdot \mathsf{mrw}} = 2$, as observed in 4.14.

6. Time and space complexity

In this section, we study the time and space complexity of computing the rank of a zero-one matrix monoid and finding a matrix of minimum rank in it. We provide two approaches with the same time complexity. The first one relies on the linear-algebraic tools developed in section 4. It turns out to have smaller space complexity than the second, combinatorial, approach, but is limited to only computing the rank. This is due to the fact that we never explicitly construct a maximal column in this approach, which is required in order to find a matrix of minimum rank by 4.1. Moreover, it is not

[4] In [KMW17, Kie20] only membership in NC is claimed, but the bottleneck computations are matrix powering and rank computation, which can in fact be done in $\mathrm{DET} \subseteq \mathsf{NC}^2$; see [Coo85]. The computations in [Kie20, Proposition 5.2] are on polynomially larger matrices, but this does not impact the membership in NC^2, as $\log^2(poly(n)) = O(\log^2(n))$.

known if one can find a maximal column in NC. In contrast, the combinatorial approach explicitly constructs a matrix of minimum rank step by step. In a way, this is exactly the reason why it requires more space to achieve the same time complexity: in the proof of 6.9, we have to precompute a linear number of matrices (one for each step), since we do not know in advance which matrices we will need and since computing them "on the fly" would have a higher time complexity. In the total DFAs case, where transition matrices require only linear amount of space and can be multiplied in linear time, the combinatorial approach becomes much more efficient, and outmatches its linear-algebraic counterpart by a factor of the alphabet size.

The three main results of this section are as follows.

Theorem 6.1. *The (real) rank of an n-state UFA over an alphabet of size m can be computed in $\mathcal{O}(mn^4)$ time and $\mathcal{O}(n^2)$ space.*

Theorem 6.2. *A matrix of minimum (real) rank in an n-state UFA over an alphabet of size m can be found in $\mathcal{O}(mn^4)$ time and $\mathcal{O}(n^3)$ space.*

Theorem 6.3. *A matrix of minimum (real) rank in an n-state total DFA over an alphabet of size m can be found in $\mathcal{O}(n^3 + mn^2)$ time and $\mathcal{O}(n^2)$ space.*

Until the end of the section, fix a strongly connected complete UFA $\mathcal{A} = (Q, \Sigma, \Delta)$. Denote $n = |Q|$, $m = |\Sigma|$. subsection 4.6 shows that strong connectivity can be assumed without loss of generality.

6.1. **Square automaton and square digraph.** We will need the construction of the square automaton of an NFA. The *square automaton* $\mathcal{A}^{(2)} = (Q^{(2)}, \Sigma, \Delta^{(2)})$ of $\mathcal{A}$ is defined as follows. Let $Q^{(2)} = \{(p, q) \mid p, q \in Q\}$, and for $p, q \in Q$ and $a \in \Sigma$, the transitions are defined component-wise, that is,

$$\Delta^{(2)} = \{((p, q), a, (p', q')) \in Q^{(2)} \times \Sigma \times Q^{(2)} \mid (p, a, p'), (q, a, q') \in \Delta\}.$$

Note that the square automaton of a total DFA is also a total DFA.

We call states of the form (q, q) in $\mathcal{A}^{(2)}$ *singletons*. Observe that the restriction of $\mathcal{A}^{(2)}$ to singletons is equal to $\mathcal{A}$. We denote by $G^{(2)} = (V^{(2)}, E^{(2)})$ the underlying digraph of $\mathcal{A}^{(2)}$ obtained by forgetting the labels of the transitions. Note that $|E^{(2)}| = \mathcal{O}(mn^4)$, and there exists an infinite series of complete UFAs over a two-letter alphabet with $|E^{(2)}| = \Theta(n^4)$ [KW19a, Appendix A]. If $\mathcal{A}$ is a total DFA, then $|E^{(2)}| = mn^2$.

6.2. **Minimum rank in $\mathcal{O}(mn^4)$ time.** We now perform the steps of the linear-algebraic algorithm described in section 5, but implement them efficiently in terms of time and space complexity.

Lemma 6.4. *For a state p, the set $\mathsf{Mer}(p)$ can be computed in $\mathcal{O}(mn^4)$ time and $\mathcal{O}(n^2)$ space.*

Proof. Perform a multi-source backwards digraph search starting from all singletons in $G^{(2)}$ and label all states $q \in Q$ such that (p, q) or (q, p) is visited during this search. This search can be performed in time linear in the number of edges of $|E^{(2)}|$, and $|E^{(2)}| = \mathcal{O}(mn^4)$. Observe that only the vertices of this digraph have to be stored in the memory explicitly, since the edges can be computed on the fly from the input without increasing the time complexity, hence the space complexity of the algorithm is $\mathcal{O}(n^2)$. □

Lemma 6.5. *A maximal pseudo-column can be found in $\mathcal{O}(mn^4)$ time and $\mathcal{O}(n^2)$ space.*

Proof. To use 4.20 we first need to set up the linear system of equations described there. The average matrix $\overline{A}$ can be computed in time $\mathcal{O}(mn^2)$. The weight vectors $\alpha, \beta \in \mathbb{Q}^Q$ can then be computed in time $\mathcal{O}(n^3)$ by solving a system of linear equations. Then, using 6.4, we compute in $\mathcal{O}(mn^4)$ time the set $\mathsf{Mer}(p)$ for some state p. We also need to compute a basis of the vector space U defined in subsection 4.5. As in the proof of 5.2, we compute a basis of $U = U' = \langle \alpha^T M'(w) \mid w \in \Sigma^+ \rangle$, which is the smallest vector space that contains $\alpha^T M(a)$ for all $a \in \Sigma$ and is closed under post-multiplication with $M(a)$ for all $a \in \Sigma$. This can be done in $\mathcal{O}(mn^3)$ time and $\mathcal{O}(n^2)$ space using a worklist algorithm and keeping a basis in echelon form using Gaussian

elimination, as described, e.g., in [Kie20, Section 2]. Finally, we solve the system of linear equations from 4.20, which can be done in $\mathcal{O}(n^3)$ time. Each step requires at most $\mathcal{O}(n^2)$ space. □

By 4.18, we thus get that mcw and mrw can be computed in $\mathcal{O}(mn^4)$ time and $\mathcal{O}(n^2)$ space, which together with 4.10 proves 6.1. We remark that for total DFAs the proof of 6.5 gives $\mathcal{O}(mn^3)$ time for computing the rank. The combinatorial algorithm provided below will improve it to $\mathcal{O}(n^3 + mn^2)$ (see subsection 6.5), while additionally finding a matrix of minimum rank.

6.3. **Efficiently constructing a maximal column.** We now consider a more general problem of finding a matrix of minimum rank in a zero-one matrix monoid. As mentioned above, by 4.1 we have to explicitly construct a maximal column for that, which turns out to be a more difficult task in terms of the time complexity. We will see in the next subsection that we only need one maximal column (together with a word constructing it) to get a matrix of minimum rank. The goal of this subsection is thus to show how to efficiently compute a short representation of a word constructing a maximal column, since the word itself may be longer than the time complexity we are aiming for. We do so by reusing repeating subwords in such a word, and describing their occurrences with a straight line program.

In [Epp90, Section 5], an algorithm for constructing a word of rank one in total DFAs in $\mathcal{O}(m^3 + mn^2)$ time and $\mathcal{O}(n^2)$ space was suggested. Our approach for finding a maximal column and a word constructing it follows a similar direction, with a few key differences. Firstly, we observe that it is enough to only compute words merging states with one chosen state p, instead of finding all pairs of mergeable states. This both simplifies the algorithm (in [Epp90] an additional step is required to decrease the space complexity from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^2)$, which we get for free) and allows to present our results in terms of straight line programs, giving a better insight into the regularities present in the constructed word of minimum rank.

We define set straight line programs (set-SLPs), which are just SLPs with multiple initial symbols, and thus encode a set of words instead of one word. Formally, a *set-SLP* is a tuple $(\mathcal{V}, \Sigma, R, \mathcal{S})$, where $\mathcal{V}$ and Σ are disjoint finite sets of *nonterminal* and *terminal* symbols respectively, $R\colon \mathcal{V} \to (\mathcal{V} \cup \Sigma)^*$ is a function defining a derivation rule for each nonterminal symbol, and $\mathcal{S} \subseteq \mathcal{V}$ is a set of *initial* symbols. For $v \in \mathcal{V}$, we write $R(v)$ as $v \to w$ with $w \in (\mathcal{V} \cup \Sigma)^*$, and we call v and w the left- and right-hand sides of this derivation rule respectively. The *length* of a set-SLP is the total length of the right-hand sides of all the derivation rules. The semantics of a set-SLP is defined as follows. Given an initial symbol $s \in \mathcal{S}$, we recursively replace each symbol in $R(s)$ with the right-hand side of its derivation rule until we obtain a word over Σ, which is called *the word encoded by* s. We require that each initial symbol produces a unique word over Σ as a result of such derivation. Namely, we require that there exists a total linear order $\leq$ on the set $\mathcal{V}$ such that for all v with $v \to w$, w does not contain $v' \in \mathcal{V}$ with $v' \leq v$. The (multi-)set of words encoded by all initial symbols is called *the set of words encoded by* a set-SLP.

Example 6.6. Consider a set-SLP $(\{w_1, w_3, w_5, u_1, u_2, u_3\}, \{a, b\}, R, \{w_1, w_3, w_5\})$ with

$$w_1 \to u_1, w_3 \to u_3u_2, w_5 \to u_2, u_1 \to aab, u_2 \to aab, u_3 \to aaba.$$

This set-SLP encodes the (multi-)set $\{aab, aabaaab, aab\}$, and illustrates the reason why we are using set-SLPs: they allow to construct sets of words out of smaller "pieces" without having to explicitly repeat these "pieces" multiple times (in our example, we are reusing u_2). Note that the set-SLPs that we construct below only encode sets of words whose total length is polynomial in the size of the set-SLPs.

Lemma 6.7. *Given a state p, a set-SLP of length $\mathcal{O}(n^2)$ defining a set $\{w_q \mid q \in \mathsf{Mer}(p)\}$, where w_q is a word with $p \in p \cdot w_q$ and $p \in q \cdot w_q$, can be computed in $\mathcal{O}(mn^4)$ time and $\mathcal{O}(n^2)$ space.*

Proof sketch. Call vertices (p, q) with $q \in \mathsf{Mer}(p)$ *merging*. The idea is to construct, by a digraph search of $G^{(2)}$, a directed tree T rooted in (p, p) and containing a path from each merging vertex to the root, and then use the joint subpaths of these paths in the tree to obtain a short set-SLP describing these paths. See Figure 5 for an example. □

Proof of 6.7. Call vertices (p, q) with $q \in \mathsf{Mer}(p)$ *merging*. The idea is to construct a directed tree rooted in (p, p) and containing paths from each merging vertex to the root, and then use the joint subpaths of these paths in the forest to obtain a short set-SLP description of these paths.

First, by performing a backwards digraph search in $G^{(2)}$ starting from (p, p), find a subgraph T of $G^{(2)}$ with the following properties (see Figure 5 for an illustration):

- T is a directed tree directed towards its root (p, p);
- if $q \in \mathsf{Mer}(q)$, then T contains (p, q) or (q, p);
- every leaf of T is a merging vertex, and these are the only leaves in T.

Call a directed path ρ in T a *maximal branch* if all its edges belong to T, every vertex of ρ except the last one has outdegree exactly one in T, and merging vertices only occur as the first or the last vertex in ρ. Intuitively, maximal branches are paths in T between each pair of consecutive branching, leaf, root or merging vertices of T. See Figure 5 for an example. Clearly, computing T and then $\rho_1, \dots, \rho_k$ can be done in time linear in $|E^{(2)}|$.

Let $w_1, \dots, w_k$ be the words labeling the paths $\rho_1, \dots, \rho_k$. By going backwards from the root of T, we can construct a set-SLP for the required words by expressing them as concatenations of $w_1, \dots, w_k$. This can easily be done in time linear in $|E^{(2)}|$.

It remains to estimate the length of the obtained set-SLP. Since T is a tree, the total length of all words $w_1, \dots, w_k$ is at most $|V^{(2)}| = n^2$. The number of maximal branches k is $\mathcal{O}(n)$. Hence, the length of the obtained set-SLP is $\mathcal{O}(n^2)$. □

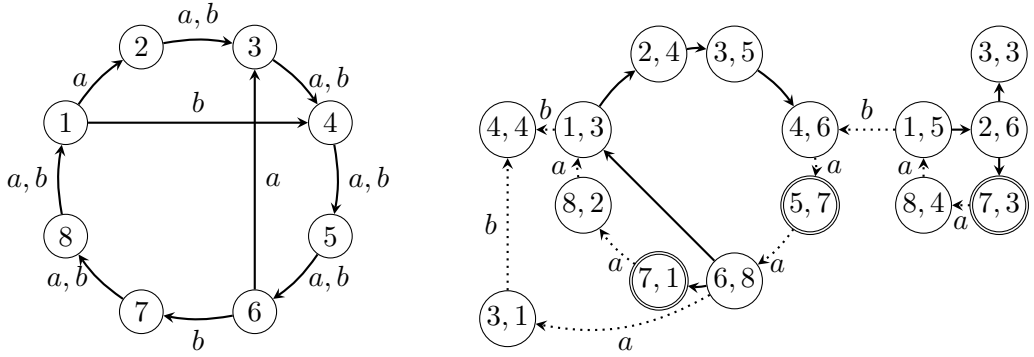


FIGURE 5. An example of $\mathcal{A}$ (left), and a part of the underlying digraph $G^{(2)}$ of its square automaton (right). Merging vertices are doubly circled. The edges of T for $p = 7$ are represented by dotted edges, and these edges are labelled with one of the letters labelling the corresponding transition in $\mathcal{A}^{(2)}$. Furthermore, we have $\rho_1 = (7, 1) \to (8, 2) \to (1, 3) \to (4, 4)$, $\rho_2 = (5, 7) \to (6, 8) \to (3, 1) \to (4, 4)$, $\rho_3 = (7, 3) \to (8, 4) \to (1, 5) \to (4, 6) \to (5, 7)$. A set-SLP encoding the labels of these path is presented in 6.6, with u_i labelling the path ρ_i, $i \in \{1, 2, 3\}$.

To construct a maximal column more efficiently, we will use [CPRvdM26, Corollary 11].

Proposition 6.8 [CPRvdM26]**.** *Given $M_1, M_2 \in \{0,1\}^{n\times n}$ such that $M_1M_2 \in \{0,1\}^{n\times n}$, one can compute M_1M_2 in $\mathcal{O}(n^2)$ time.*

Since in the remainder of this section we are dealing only with matrices from a fixed zero-one matrix monoid, 6.8 means that we can always perform matrix multiplication in quadratic time.

Lemma 6.9. *For a given state p of $\mathcal{A}$, an SLP of length $\mathcal{O}(n^2)$ encoding a word w such that $[w \cdot p]$ is a maximal column can be computed in $\mathcal{O}(mn^4)$ time and $\mathcal{O}(n^3)$ space.*

Proof sketch. Compute the matrices of the words encoded by the set-SLP from 6.7. We rely on the property, already used in some form in [Car88], that if for all $q \in \mathsf{Mer}(p)$, $q \neq p$, the vector $[w \cdot q]$ is zero, then $[w \cdot p]$ is a maximal column. To construct a word with this property, we iteratively concatenate the words w_q depending on nonzero columns in the matrix in each iteration. The number of iterations is bounded by n. □

Proof of 6.9. By 6.7, we can compute in $\mathcal{O}(mn^4)$ time a set-SLP of length $\mathcal{O}(n^2)$ defining a set $\{w_q \mid q \in \mathsf{Mer}(p)\}$, where w_q is a word with $p \in p \cdot w_q$ and $p \in q \cdot w_q$. For each $q \in \mathsf{Mer}(p)$, we then compute $M(w_q)$, which in total requires $\mathcal{O}(n^2)$ matrix multiplications and $\mathcal{O}(n^3)$ space.

Perform now the following algorithm that iteratively constructs SLPs for words w_i and $M(w_i)$ for $i \geq 0$. Let w_0 be the empty word. Assume that an SLP for w_{i-1} and $M(w_{i-1})$ are already constructed. If there is a state $q \in \mathsf{Mer}(p)$, $q \neq p$, such that $[w_{i-1} \cdot q] = M(w_{i-1})[q]$ is not the zero vector, take $w_i = w_{i-1}w_q$ and compute $M(w_i)$, otherwise stop and output w_{i-1}. Clearly, each step of the algorithm only requires a constant number of matrix multiplications.

The algorithm clearly terminates in at most $|\mathsf{Mer}(p)|$ steps, since each iteration decrements by at least one the number of states $q \in \mathsf{Mer}(p)$ such that $[w_i \cdot q]$ is not the zero vector. It is also easy to see that the constructed SLP for w has length $\mathcal{O}(n^2)$.

Assume now that $[w \cdot p]$ is not a maximal column. Let $C \subseteq Q$ be such that $[C] \geq [w \cdot p]$ is a maximal column. By definition of $\mathsf{Mer}(p)$, $C \subseteq \mathsf{Mer}(p)$, hence for every $q \in C$, $q \neq p$, the vector $[w \cdot q]$ is zero. Since $[C] \geq [w \cdot p]$, $p \in C$. Thus we get that $M(w)[C] = \sum_{q\in C}[w \cdot q] = [w \cdot p]$. By 4.8, $M(w)[C]$ is a maximal column, hence $[w \cdot q]$ must also be a maximal column. □

6.4. **Finding a matrix of minimum rank.** We now use the results of the previous section to construct a matrix of minimum rank. The key idea is as follows: since the set of maximal columns is stable under left multiplication by matrices from the monoid, we can iteratively make each column of the matrix maximal or zero by, intuitively, applying the same word (together with a short "reachability" word) to a state in each column. This simple observation significantly decreases the time complexity of our algorithm compared to the naive implementation of the algorithm from [Car88] constructing a word of minimum rank. Indeed, in the approach of [Car88], the word is constructed letter by letter, and requires to know the result of applying the last letter at every step. Since the constructed word has length $\mathcal{O}(rn^3) = \mathcal{O}(n^4)$, where n is the number of states and r is the rank, this results in $\mathcal{O}(n^{4+\omega})$ time complexity. By efficiently constructing only one maximal column (as described in the previous section) and reusing it for the whole set of states (as described in this section), we decrease the time complexity to $\mathcal{O}(n^{2+\omega})$.

Proposition 6.10. *An SLP of length $\mathcal{O}(n^2)$ encoding a word w of minimum rank can be computed in $\mathcal{O}(mn^4)$ time and $\mathcal{O}(n^3)$ space.*

Proof sketch. Compute the matrix of the word w encoded by the SLP from 6.9. Iteratively, for each $q \in Q$, concatenate w with a word of length at most n mapping p to a state corresponding to a nonzero element of the row in the current iteration. Denote by w_n the resulting word, which has the property that all nonzero columns of $M(w_n)$ are maximal. Symmetrically compute w'_n for rows. Then by 4.9 the word $w_nw'_nw_nw'_n$ matrix has minimum rank. □

Proof of 6.10. By 6.9, for a given state p of $\mathcal{A}$, we can compute in $\mathcal{O}(mn^4)$ time an SLP of length $\mathcal{O}(n^2)$ defining a word w such that $[w \cdot p]$ is a maximal column. Compute $M(w)$, which can be done in $\mathcal{O}(n^4)$ time.

Let $Q = \{q_1, \dots, q_n\}$. Define $v_0 = \epsilon$. Clearly, $M(v_0)$ is the identity matrix. For each $1 \leq i \leq n$, perform the following algorithm. If $[v_{i-1} \cdot q_i]$ is the zero vector, take $v_i = v_{i-1}$ and go the the next step. Otherwise, find q such that $q \in v_{i-1} \cdot q_i$, and find a word $u_{p \to q}$ of length at most $n-1$ mapping p to q, and compute $M(u_{p \to q})$, which can be done by $\mathcal{O}(n)$ matrix multiplications. Take $w_i = w u_{p \to q} w_{i-1}$, and compute $M(w_i)$, which requires a constant number of matrix multiplications. Go to the next step of the algorithm.

Observe that after the ith step of the algorithm, $[v_j \cdot q_j]$ is a maximal column. Indeed, $[v_i \cdot q_i] = [w \cdot p]$, and for all $j < i$, $[v_j \cdot q_j]$ is a result of left multiplication of a maximal column by a matrix from the monoid, which is a maximal column by 4.8. Moreover, the obtained SLP of w_n has length $\mathcal{O}(n^2)$.

It remains to construct symmetrically w'_n for rows instead of columns. Then each column and each row of $M(w_n w'_n)$ is either maximal or zero, and by 4.9 the word $w_n w'_n w_n w'_n$ matrix has minimum rank. □

Given an SLP of length $\mathcal{O}(n^2)$ encoding a word w, we can compute the matrix of w by computing the matrices of words occurring in the derivation of w from bottom to top in time $\mathcal{O}(n^4)$. Thus we prove 6.2. We also get the following result, which can be seen as a proof of a very weak version of the Černý conjecture generalised from rank one words in total DFAs to minimum rank words in complete UFAs.

Theorem 6.11. *For every n-state complete UFA, there exists an SLP of length $\mathcal{O}(n^2)$ encoding a word of minimum rank.*

We remark that the length of the word encoded by the constructed SLP asymptotically matches the best known upper bound for words of minimum rank: $\mathcal{O}(n^4)$ for complete UFAs [Car88] and $\mathcal{O}(n^3)$ for total DFAs [KRS87]. In particular, one can efficiently compute words of minimum rank within these bounds.

6.5. **Total DFAs.** For total DFAs, we follow the same algorithms as in the proof of 6.2, but exploit the fact that elementary matrix operations can be performed more efficiently. Namely, if $\mathcal{A}$ is a total DFA, then each word defines a transformation on Q. By storing matrices of words as transformations, we get that matrix multiplication can be performed in $\mathcal{O}(n)$ time, and each matrix requires $\mathcal{O}(n)$ space. Moreover, we have $|E^{(2)}| = mn^2$. By taking these improvements into account, we get the proof of 6.3.

7. Algebraic synchronisation criterion

The rank of M can be viewed as a combinatorial property, in that matrix multiplication is not commutative, and even the rank of a matrix product can depend on the order of the multiplied matrices. Extending the results of section 4 but aiming at a more structural result, we address a more general question in this section: is there a specific vector space, perhaps a joint invariant subspace of the generating matrices, whose dimension tells us something about the rank of M?

In total DFAs, every row $[q]^T$ is maximal, so $\langle \mathsf{MRow} \rangle = \mathbb{R}^Q$. In [BS16, Criterion 1], the following result was proved. A different proof was independently and concurrently provided in [VP15, §8], see also [Pro21][5].

Theorem 7.1 (Algebraic synchronization criterion for total DFAs)**.** *If M is a total DFA, we have $\langle \alpha^T M(w) \mid w \in \Sigma^* \rangle = \mathbb{R}^Q$ if and only if $r = 1$.*

[5][Pro21, Theorem 2] states that if the rank of a total DFA is greater than one, then there exists a non-trivial joint invariant subspace of its generating matrices. However, such a subspace exists for all total DFAs regardless of their rank [PV17b, Corollary 4]. Hence, to characterise total DFAs of rank greater than one, we have to consider the existence of some specific joint invariant linear subspace, for example the one in the statement of 7.1.

It is thus reasonable to ask if this statement can be generalised to the case where $r > 1$ and M is an arbitrary unambiguous monoid morphism. The theorem below provides such a generalisation, using the results obtained above. In particular, it implies that for total DFAs $\dim \langle \alpha^T M(w) \mid w \in \Sigma^* \rangle \leq n - r + 1$. To be consistent with the previous results, we formulate the statements for columns, but an analogous symmetric version for rows, involving $\langle \alpha^T M(w) \mid w \in \Sigma^* \rangle$ and $\langle \mathsf{MRow} \rangle$ instead of, respectively, V and $\langle \mathsf{MCol} \rangle$ also holds true.

Theorem 7.2. *Define* $V := \langle M(w)\beta \mid w \in \Sigma^* \rangle$. *We have:*

(a) $V \subseteq \langle \mathsf{MCol} \rangle$.
(b) $\dim V + r - 1 \leq \dim \langle \mathsf{MCol} \rangle$.
(c) $V = \langle \mathsf{MCol} \rangle$ *if and only if* $r = 1$.

Proof. Towards item (a), it suffices to show that $\langle \mathsf{MCol} \rangle^\perp \subseteq V^\perp$. Let $x^\top \in \langle \mathsf{MCol} \rangle^\perp$. For every word $u \in \Sigma^*$ of minimum rank, by 4.1, each column of $M(u)$ is in $\langle \mathsf{MCol} \rangle$. Thus, $x^\top M(u)\beta = 0$ holds for all $u \in \Sigma^*$ of minimum rank. By 4.12, it follows that $x^\top M(w)\beta = 0$ holds for all $w \in \Sigma^*$, i.e., $x^\top \in V^\perp$.

Towards item (b), let $u \in \Sigma^*$ be of minimum rank. By 4.1, there are r maximal columns $[C_1], \ldots, [C_r]$ and r maximal rows $[R_1]^T, \ldots, [R_r]^T$ such that $M(u) = \sum_{i=1}^r [C_i][R_i]^T$. We conclude from item (a) that $\langle V, [C_1], \ldots, [C_r] \rangle \subseteq \langle \mathsf{MCol} \rangle$. Hence, it suffices to show that $\dim V + r - 1 = \dim \langle V, [C_1], \ldots, [C_{r-1}] \rangle$.

We have

$$M(uu) \;=\; \left(\sum_{i=1}^{r} [C_i][R_i]^T \right) \left(\sum_{j=1}^{r} [C_j][R_j]^T \right) \;=\; \sum_{1 \leq i,j \leq r} [C_i][R_i]^T [C_j][R_j]^T \,.$$

By unambiguousness, $[R_i]^T[C_j] \leq 1$ for all $1 \leq i, j \leq r$. For each i there is at most one j with $[R_i]^T[C_j] = 1$ as otherwise $[R_j]^T + [R_{j'}]^T$ for some $j' \neq j$ would appear in $M(uu)$, contradicting the maximality of $[R_j]^T$. On the other hand, for each i there is at least one j with $[R_i]^T[C_j] = 1$ as otherwise $M(uu)$ has only $[C_{i'}]$ with $i' \neq i$ as nonzero columns, contradicting the fact that $M(uu)$ has rank r. Thus, for each i there is exactly one j with $[R_i]^T[C_j] = 1$. With a symmetric argument, for each j there is exactly one i with $[R_i]^T[C_j] = 1$. It follows that there is permutation $\pi : \{1, \ldots, r\} \to \{1, \ldots, r\}$ such that $[R_i]^T[C_j] = 1$ if and only if $i = \pi(j)$.

To show that $\dim V + r - 1 = \dim \langle V, [C_1], \ldots, [C_{r-1}] \rangle$, it suffices to show for each $j \in \{1, \ldots, r-1\}$ that $[C_j] \notin \langle V, [C_1], \ldots, [C_{j-1}] \rangle$. To this end, let $1 \leq j < r$ and define $x^T := [R_{\pi(j)}]^T - [R_{\pi(r)}]^T$. Since $[R_{\pi(j)}]^T$ and $[R_{\pi(r)}]^T$ are maximal rows, for all $w \in \Sigma^*$ also $[R_{\pi(j)}]^T M(w)$ and $[R_{\pi(r)}]^T M(w)$ are maximal rows, implying that $x^T M(w)\beta = \mathsf{mrw} - \mathsf{mrw} = 0$; i.e., x^T is in the orthogonal complement of V. Further, for all $i \in \{1, \ldots, j-1\}$ we have $x^T[C_i] = [R_{\pi(j)}]^T[C_i] - [R_{\pi(r)}]^T[C_i] = 0 - 0 = 0$; i.e., x^T is orthogonal also to $[C_i]$. But x^T is not orthogonal to $[C_j]$, as $x^T[C_j] = [R_{\pi(j)}]^T[C_j] - [R_{\pi(r)}]^T[C_j] = 1 - 0 = 1$. This completes the proof of item (b).

Towards item (c), if $r > 1$ we have $V \subsetneq \langle \mathsf{MCol} \rangle$ by item (b). Let $r = 1$. By item (a) we have $V \subseteq \langle \mathsf{MCol} \rangle$. Towards the opposite inclusion, let $y \in \mathsf{MCol}$. Since $r = 1$, there is $w \in \Sigma^*$ such that y is the only nonzero column of $M(w)$. Thus, y is a multiple of $M(w)\beta$. Hence, $y \in V$. □

Remark 7.3. The inequality in 7.2 (b) and its row version can be both strict if $r > 1$, even in total DFAs. In 4.14, V is spanned by $\begin{pmatrix} 1 & 1 & 1 & 1 \end{pmatrix}^T$, but

$$\dim \langle \mathsf{MCol} \rangle = 3 > 1 + 2 - 1 = \dim V + r - 1.$$

For the row version, $\langle \alpha^T M(w) \mid w \in \Sigma^* \rangle$ is spanned by $\begin{pmatrix} 1 & 0 & 1 & 0 \end{pmatrix}$ and $\begin{pmatrix} 0 & 1 & 0 & 1 \end{pmatrix}$, but

$$\dim \langle \mathsf{MRow} \rangle = 4 > 2 + 2 - 1 = \dim \langle \alpha^T M(w) \mid w \in \Sigma^* \rangle + r - 1.$$

8. Conclusions and open problems

We list a few open questions that follow directly from our work.

- In [Epp90], it was asked if a word of rank one for a total DFA can be found in NC. Similarly, can a matrix of minimum rank for a total DFA be computed in NC?
- Given an unambiguous morphism $M\colon \Sigma \to \{0,1\}^{Q\times Q}$ and a vector $\alpha \in \mathbb{Q}^Q_{>0}$, can a basis of $\langle \alpha^T M(w) \mid w \in \Sigma^*\rangle$ be computed faster than in $\mathcal{O}(|Q|^3)$ time? This would improve algorithms for several fundamental problems for weighted automata [Kie20]. Similarly, for total DFAs, computing a basis of U from subsection 4.5 in subcubic time (see the proof of 6.5) would allow to compute the rank of a total DFA faster than in cubic time.
- Can one decide if a total DFA has rank one in strongly subquadratic time (in the number of states)? This seems to be a major open problem in the area of synchronising automata.
- The bottleneck in the time complexity in 6.1 is the very first step, computing $\mathsf{Mer}(q)$ via digraph search in the square digraph of $\mathcal{A}$. The number of edges of this digraph can be quadratic in the number of its vertices [KW19a, Appendix A], hence of order $|Q|^4$. Can $\mathsf{Mer}(q)$ be computed faster than in time $\mathcal{O}(|Q|^4)$? Very little seems to be known about general properties of square automata of DFAs or UFAs.
- Finally, a natural continuation of this work is to consider the minimum nonzero rank of zero-one matrix monoids. It is equal to the rank of each nonzero matrix in the 0-minimal ideal of the monoid. It is not known how to compute it in NC even for total DFAs. The main motivation once again comes from the degree of variable-length codes, see [BPR10, Chapter 9] for more details.

References

[AA13] Yu. A. Al'pin and V. S. Al'pina. Combinatorial properties of irreducible semigroups of nonnegative matrices. *Journal of Mathematical Sciences*, 191(1):4–9, 2013. doi:10.1007/s10958-013-1295-8.

[AA19] Yu. A. Al'pin and V. S. Al'pina. A new proof of the Protasov-Voynov theorem on semigroups of nonnegative matrices. *Mathematical Notes*, 105(5):805–811, 2019. doi:10.1134/S0001434619050183.

[AB09] Sanjeev Arora and Boaz Barak. *Computational complexity: a modern approach*. Cambridge University Press, 2009. doi:10.1017/CBO9780511804090.

[AM10] Farid M. Ablayev and Ernst W. Mayr, editors. *Computer Science - Theory and Applications, 5th International Computer Science Symposium in Russia, CSR 2010, Kazan, Russia, June 16-20, 2010. Proceedings*, volume 6072 of *Lecture Notes in Computer Science*. Springer, 2010.

[APMS+99] Giorgio Ausiello, M. Protasi, A. Marchetti-Spaccamela, G. Gambosi, P. Crescenzi, and V. Kann. *Complexity and Approximation: Combinatorial Optimization Problems and Their Approximability Properties*. Springer-Verlag, Berlin, Heidelberg, 1st edition, 1999.

[APY91] Esther M. Arkin, Christos H. Papadimitriou, and Mihalis Yannakakis. Modularity of cycles and paths in graphs. *Journal of the ACM*, 38(2):255–274, 1991. doi:10.1145/103516.103517.

[AS09] Jorge Almeida and Benjamin Steinberg. Matrix mortality and the Černý-pin conjecture. In Volker Diekert and Dirk Nowotka, editors, *Developments in Language Theory, 13th International Conference, DLT 2009, Stuttgart, Germany, June 30 - July 3, 2009. Proceedings*, volume 5583 of *Lecture Notes in Computer Science*, pages 67–80. Springer, 2009. doi:10.1007/978-3-642-02737-6_5.

[Ata82] Mikhail J. Atallah. Finding the cyclic index of an irreducible, nonnegative matrix. *SIAM Journal on Computing*, 11(3):567–570, 1982. doi:10.1137/0211047.

[AV04] Dimitry S. Ananichev and Mikhail V. Volkov. Synchronizing monotonic automata. *Theoretical Computer Science*, 327(3):225–239, 2004.

[AV05] Dimitry S. Ananichev and Mikhail V. Volkov. Synchronizing generalized monotonic automata. *Theoretical Computer Science*, 330(1):3–13, 2005.

[BBC+96] László Babai, Robert Beals, Jin-yi Cai, Gábor Ivanyos, and Eugene M. Luks. Multiplicative equations over commuting matrices. In *Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms*, SODA '96, pages 498–507, USA, 1996. Society for Industrial and Applied Mathematics.

[BCKP08a] Marie-Pierre Béal, Eugen Czeizler, Jarkko Kari, and Dominique Perrin. Unambiguous automata. *Mathematics in Computer Science*, 1(4):625–638, 2008. doi:10.1007/S11786-007-0027-1.

[BCKP08b] Marie-Pierre Béal, Eugen Czeizler, Jarkko Kari, and Dominique Perrin. Unambiguous automata. *Mathematics in Computer Science*, 1:625–638, 2008. doi:10.1007/s11786-007-0027-1.

[Bea88a] Martin Beaudry. Membership testing in commutative transformation semiuniversit¨at stuttgartgroups. *Information and Computation*, 79(1):84–93, 1988. doi:10.1016/0890-5401(88)90018-1.

[Bea88b] Martin Beaudry. Membership testing in transformation monoids. *PhD thesis, McGill University, Montreal, Quebec*, 1988.

[Bea94] Martin Beaudry. Membership testing in threshold one transformation monoids. *Information and Computation.*, 113(1):1–25, 1994. doi:10.1006/INCO.1994.1062.

[Ber84] Stuart J. Berkowitz. On computing the determinant in small parallel time using a small number of processors. *Information Processing Letters*, 18(3):147–150, 1984. doi:10.1016/0020-0190(84)90018-8.

[Ber14] Mikhail V. Berlinkov. On two algorithmic problems about synchronizing automata (short paper). In Arseny M. Shur and Mikhail V. Volkov, editors, *Developments in Language Theory – 18th International Conference, DLT 2014, Ekaterinburg, Russia, August 26-29, 2014. Proceedings*, volume 8633 of *Lecture Notes in Computer Science*, pages 61–67. Springer, 2014. doi:10.1007/978-3-319-09698-8_6.

[BF80] J.A. Brzozowski and Faith E. Fich. Languages of R-trivial monoids. *Journal of Computer and System Sciences*, 20(1):32–49, 1980. doi:10.1016/0022-0000(80)90003-3.

[BF11] Greg Budzban and Philip Feinsilver. The generalized road coloring problem and periodic digraphs. *Applicable Algebra in Engineering, Communication and Computing*, 22:21–35, 2011. doi:10.1007/s00200-010-0135-z.

[BF19] Jens Bruchertseifer and Henning Fernau. Synchronizing series-parallel automata with loops. In Freund et al. [FHS19], pages 63–78.

[BFM23] Nathalie Bertrand, Hugo Francon, and Nicolas Markey. Synchronizing words under LTL constraints. *Information Processing Letters*, 182:106392, 2023. doi:10.1016/j.ipl.2023.106392.

[BFRS21] Mikhail V. Berlinkov, Robert Ferens, Andrew Ryzhikov, and Marek Szykuła. Synchronizing Strongly Connected Partial DFAs. In Markus Bläser and Benjamin Monmege, editors, *38th International Symposium on Theoretical Aspects of Computer Science (STACS 2021)*, volume 187 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 12:1–12:16, Dagstuhl, Germany, 2021. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.STACS.2021.12.

[BHK+20] Georgina Bumpus, Christoph Haase, Stefan Kiefer, Paul-Ioan Stoienescu, and Jonathan Tanner. On the size of finite rational matrix semigroups. In Artur Czumaj, Anuj Dawar, and Emanuela Merelli, editors, *47th International Colloquium on Automata, Languages, and Programming, ICALP 2020, July 8-11, 2020, Saarbrücken, Germany (Virtual Conference)*, volume 168 of *LIPIcs*, pages 115:1–115:13. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020. doi:10.4230/LIPICS.ICALP.2020.115.

[BHMR21] Michael Blondin, Christoph Haase, Filip Mazowiecki, and Mikhail A. Raskin. Affine extensions of integer vector addition systems with states. *Logical Methods in Computer Science*, 17(3), 2021. doi:10.46298/LMCS-17(3:1)2021.

[BJ73] Yves Balcer and Arthur F. Veinott Jr. Computing a graph's period quadratically by node condensation. *Discret. Math.*, 4(4):295–303, 1973. doi:10.1016/0012-365X(73)90166-0.

[BJG08] Jørgen Bang-Jensen and Gregory Z Gutin. *Digraphs: theory, algorithms and applications*. Springer Science & Business Media, 2008.

[BJK+05] Manfred Broy, Bengt Jonsson, Joost-Pieter Katoen, Martin Leucker, and Alexander Pretschner, editors. *Model-Based Testing of Reactive Systems, Advanced Lectures [The volume is the outcome of a research seminar that was held in Schloss Dagstuhl in January 2004]*, volume 3472 of *Lecture Notes in Computer Science*. Springer, 2005.

[BJO15] Vincent D. Blondel, Raphaël M. Jungers, and Alex Olshevsky. On primitivity of sets of matrices. *Automatica*, 61:80–88, 2015. doi:10.1016/J.AUTOMATICA.2015.07.026.

[BK08] Christel Baier and Joost-Pieter Katoen. *Principles of model checking*. MIT press, 2008.

[BKK+23] Christel Baier, Stefan Kiefer, Joachim Klein, David Müller, and James Worrell. Markov chains and unambiguous automata. *Journal of Computer and System Sciences*, 136:113–134, 2023. doi:10.1016/J.JCSS.2023.03.005.

[BKM16] Michael Blondin, Andreas Krebs, and Pierre McKenzie. The complexity of intersecting finite automata having few final states. *computational complexity*, 25(4):775–814, 2016. doi:10.1007/s00037-014-0089-9.

[BMT92] M. Beaudry, P. McKenzie, and D. Thérien. The membership problem in aperiodic transformation monoids. *Journal of the ACM*, 39(3):599–616, 1992.

[Bor77] Allan Borodin. On relating time and space to size and depth. *SIAM Journal on Computing*, 6(4):733–744, 1977. doi:10.1137/0206054.

[BP14] Marie-Pierre Béal and Dominique Perrin. A quadratic algorithm for road coloring. *Discrete Applied Mathematics*, 169:15–29, 2014. doi:10.1016/J.DAM.2013.12.002.

[BP16a] Marie-Pierre Béal and Dominique Perrin. Synchronised automata. In Valérie Berthé and Michel Rigo, editors, *Combinatorics, Words and Symbolic Dynamics*, Encyclopedia of Mathematics and its Applications, pages 213–240. Cambridge University Press, 2016. doi:10.1017/CBO9781139924733.008.

[BP16b] Marie-Pierre Béal and Dominique Perrin. Synchronised automata. In Valérie Berthé and Michel Rigo, editors, *Combinatorics, Words and Symbolic Dynamics*, Encyclopedia of Mathematics and its Applications, pages 213–240. Cambridge University Press, 2016. doi:10.1017/CBO9781139924733.008.

[BP16c] Marie-Pierre Béal and Dominique Perrin. *Synchronised Automata*, page 213–240. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 2016.

[BPR09] Jean Berstel, Dominique Perrin, and Christophe Reutenauer. *Codes and Automata*. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 2009. doi:10.1017/CBO9781139195768.

[BPR10] Jean Berstel, Dominique Perrin, and Christophe Reutenauer. *Codes and automata*, volume 129. Cambridge University Press, 2010. doi:10.1017/CBO9781139195768.

[BPS21] Paul C. Bell, Igor Potapov, and Pavel Semukhin. On the mortality problem: From multiplicative matrix equations to linear recurrence sequences and beyond. *Information and Computation*, 281:104736, 2021. doi:10.1016/J.IC.2021.104736.

[BR97] R. B. Bapat and T. E. S. Raghavan. *Nonnegative Matrices and Applications*. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 1997.

[BS16] Mikhail V. Berlinkov and Marek Szykuła. Algebraic synchronization criterion and computing reset words. *Information Sciences*, 369:718–730, 2016. doi:10.1016/J.INS.2016.07.049.

[BvzGH82] Allan Borodin, Joachim von zur Gathen, and John E. Hopcroft. Fast parallel matrix and GCD computations. *Information and Control*, 52(3):241–256, 1982. doi:10.1016/S0019-9958(82)90766-5.

[Car87] Arturo Carpi. On unambiguous reductions of monoids of unambiguous relations. *Theoretical Computer Science*, 51:215–220, 1987. doi:10.1016/0304-3975(87)90055-7.

[Car88] Arturo Carpi. On synchronizing unambiguous automata. *Theoretical Computer Science*, 60:285–296, 1988. doi:10.1016/0304-3975(88)90114-4.

[CD09] Arturo Carpi and Flavio D'Alessandro. Strongly transitive automata and the Černý conjecture. *Acta Informatica*, 46(8):591–607, 2009. doi:10.1007/S00236-009-0106-7.

[CD16] Krishnendu Chatterjee and Laurent Doyen. Computation tree logic for synchronization properties. In Ioannis Chatzigiannakis, Michael Mitzenmacher, Yuval Rabani, and Davide Sangiorgi, editors, *43rd International Colloquium on Automata, Languages, and Programming, ICALP 2016, July 11-15, 2016, Rome, Italy*, volume 55 of *LIPIcs*, pages 98:1–98:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016. doi:10.4230/LIPIcs.ICALP.2016.98.

[Čer64] Ján Černý. Poznámka k homogénnym eksperimentom s konečnými automatami. *Matematicko-fyzikalny Časopis Slovensk*, 14(3):208–215, 1964.

[Cer19] Ján Cerný. A note on homogeneous experiments with finite automata. *Journal of Automata, Languages and Combinatorics*, 24(2-4):123–132, 2019.

[Cés74] Yves Césari. Sur l'application du théorème de Suschkewitsch à l'étude des codes rationnels complets. In Jacques Loeckx, editor, *Automata, Languages and Programming, 2nd Colloquium, University of Saarbrücken, Germany, July 29 - August 2, 1974, Proceedings*, volume 14 of *Lecture Notes in Computer Science*, pages 342–350. Springer, 1974. doi:10.1007/3-540-06841-4_73.

[CFK+15] Marek Cygan, Fedor V. Fomin, Lukasz Kowalik, Daniel Lokshtanov, Dániel Marx, Marcin Pilipczuk, Michal Pilipczuk, and Saket Saurabh. *Parameterized Algorithms*. Springer, 2015.

[CGJH17] Pierre-Yves Chevalier, Vladimir V. Gusev, Raphaël M. Jungers, and Julien M. Hendrickx. Sets of stochastic matrices with converging products: Bounds and complexity. *CoRR*, abs/1712.02614, 2017. URL: http://arxiv.org/abs/1712.02614, arXiv:1712.02614.

[CHHN14] Julien Cassaigne, Vesa Halava, Tero Harju, and François Nicolas. Tighter undecidability bounds for matrix mortality, zero-in-the-corner problems, and more. *CoRR*, abs/1404.0644, 2014. arXiv:1404.0644.

[CHJ15] Pierre-Yves Chevalier, Julien M. Hendrickx, and Raphaël M. Jungers. Reachability of consensus and synchronizing automata. In *54th IEEE Conference on Decision and Control, CDC 2015, Osaka, Japan, December 15-18, 2015*, pages 4139–4144. IEEE, 2015. doi:10.1109/CDC.2015.7402864.

[CLZ00] Jin-yi Cai, Richard J. Lipton, and Yechezkel Zalcstein. The complexity of the A B C problem. *SIAM Journal on Computing*, 29(6):1878–1888, 2000. doi:10.1137/S0097539794276853.

[CMP92] Edward Y. Chang, Zohar Manna, and Amir Pnueli. Characterization of temporal property classes. In Werner Kuich, editor, *Automata, Languages and Programming, 19th International Colloquium, ICALP92, Vienna, Austria, July 13-17, 1992, Proceedings*, volume 623 of *Lecture Notes in Computer Science*, pages 474–486. Springer, 1992. doi:10.1007/3-540-55719-9_97.

[Coo85] Stephen A. Cook. A taxonomy of problems with fast parallel algorithms. *Inf. Control.*, 64(1-3):2–21, 1985. doi:10.1016/S0019-9958(85)80041-3.

[CPRvdM26] Dmitry Chistikov, Radosław Piórkowski, Neha Rino, and Brink van der Merwe. Algorithms and fine-grained complexity for nondeterministic and symmetric difference automata. *CoRR*, abs/2607.00742, 2026. URL: https://arxiv.org/abs/2607.00742, arXiv:2607.00742.

[Csa76] Laszlo Csanky. Fast parallel matrix inversion algorithms. *SIAM Journal on Computing*, 5(4):618–623, 1976. doi:10.1137/0205040.

[DBL86] *27th Annual Symposium on Foundations of Computer Science, Toronto, Canada, 27-29 October 1986*. IEEE Computer Society, 1986.

[DGDMM14] Giuseppe De Giacomo, Riccardo De Masellis, and Marco Montali. Reasoning on LTL on finite traces: Insensitivity to infiniteness. In *Proceedings of the Twenty-Eighth AAAI Conference on Artificial Intelligence*, AAAI'14, pages 1027–1033. AAAI Press, 2014.

[DGV13] Giuseppe De Giacomo and Moshe Y. Vardi. Linear temporal logic and linear dynamic logic on finite traces. In *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence*, IJCAI '13, pages 854–860. AAAI Press, 2013.

[DJL+14] Laurent Doyen, Line Juhl, Kim Guldstrand Larsen, Nicolas Markey, and Mahsa Shirmohammadi. Synchronizing words for weighted and timed automata. In Raman and Suresh [RS14], pages 121–132.

[Dre15] Frank Drewes, editor. *Implementation and Application of Automata - 20th International Conference, CIAA 2015, Umeå, Sweden, August 18-21, 2015, Proceedings*, volume 9223 of *Lecture Notes in Computer Science*. Springer, 2015.

[EKSW04] Keith Ellul, Bryan Krawetz, Jeffrey Shallit, and Ming-Wei Wang. Regular expressions: new results and open problems. *Journal of Automata, Languages and Combinatorics*, 9(2–3):233—256, 2004.

[Epp90] David Eppstein. Reset sequences for monotonic automata. *SIAM Journal on Computing*, 19(3):500–510, 1990. doi:10.1137/0219033.

[Ete97] Kousha Etessami. Counting quantifiers, successor relations, and logarithmic space. *Journal of Computer and System Sciences*, 54(3):400–411, 1997. doi:10.1006/JCSS.1997.1485.

[FGH+19] Henning Fernau, Vladimir V. Gusev, Stefan Hoffmann, Markus Holzer, Mikhail V. Volkov, and Petra Wolf. Computational complexity of synchronization under regular constraints. In Rossmanith et al. [RHK19], pages 63:1–63:14.

[FHS19] Rudolf Freund, Markus Holzer, and José M. Sempere, editors. *Eleventh Workshop on Non-Classical Models of Automata and Applications, NCMA 2019, Valencia, Spain, July 2-3, 2019*. Österreichische Computer Gesellschaft, 2019.

[FHV15] Henning Fernau, Pinar Heggernes, and Yngve Villanger. A multi-parameter analysis of hard problems on deterministic finite automata. *Journal of Computer and System Sciences*, 81(4):747–765, 2015.

[Fle19] Lukas Fleischer. Algorithms and complexity results for finite semigroups. *PhD thesis, Universität Stuttgart*, 2019.

[Fle22] Lukas Fleischer. The cayley semigroup membership problem. *Theory of Computing*, 18(8):1–18, 2022. doi:10.4086/toc.2022.v018a008.

[FMV13] Fedor M. Fominykh, Pavel V. Martyugin, and Mikhail V. Volkov. P(L)Aying for synchronization. *International Journal of Foundations of Computer Science*, 24(6):765–780, 2013.

[Fri90] Joel Friedman. On the road coloring problem. *Proceedings of the American Mathematical Society*, 110(4):1133–1135, 1990. doi:10.1007/s11856-009-0062-5.

[Fro97] V. Froidure. Ranks of binary relations. *Semigroup Forum*, 54:381–401, 1997. doi:10.1007/BF02676619.

[GGJ18] Balázs Gerencsér, Vladimir V. Gusev, and Raphaël M. Jungers. Primitive sets of nonnegative matrices and synchronizing automata. *SIAM Journal on Matrix Analysis and Applications*, 39(1):83–98, 2018. doi:10.1137/16M1094099.

[GHR95] Raymond Greenlaw, H James Hoover, and Walter L Ruzzo. *Limits to parallel computation: P-completeness theory*. Oxford University Press, 1995.

[GIN09] Zsolt Gazdag, Szabolcs Iván, and Judit Nagy-György. Improved upper bounds on synchronizing nondeterministic automata. *Information Processing Letters*, 109(17):986–990, 2009. doi:10.1016/J.IPL.2009.05.007.

[GK95] Pavel Goralčík and Václav Koubek. Rank problems for composite transformations. *International Journal of Algebra and Computation*, 05(03):309–316, 1995. doi:10.1142/S0218196795000185.

[Gol08] Oded Goldreich. *Computational Complexity: A Conceptual Perspective*. Cambridge University Press, 2008. doi:10.1017/CBO9780511804106.

[GP16] Vladimir V. Gusev and Elena V. Pribavkina. On synchronizing colorings and the eigenvectors of digraphs. In Piotr Faliszewski, Anca Muscholl, and Rolf Niedermeier, editors, *41st International Symposium on Mathematical Foundations of Computer Science, MFCS 2016, August 22-26, 2016 - Kraków, Poland*, volume 58 of *LIPIcs*, pages 48:1–48:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2016. doi:10.4230/LIPICS.MFCS.2016.48.

[Haj76] J. Hajnal. On products of non-negative matrices. *Mathematical Proceedings of the Cambridge Philosophical Society*, 79(3):521–530, 1976. doi:10.1017/S030500410005252X.

[Har01] Darald J Hartfiel. *Nonhomogeneous Matrix Products*. WORLD SCIENTIFIC, 2001. doi:10.1142/4707.

[HJ18] Markus Holzer and Sebastian Jakobi. On the computational complexity of problems related to distinguishability sets. *Information and Computation*, 259(2):225–236, 2018. doi:10.1016/J.IC.2017.09.003.

[HL66] B. R. Heap and M. S. Lynn. The structure of powers of nonnegative matrices i. the index of convergence. *SIAM Journal on Applied Mathematics*, 14(3):610–639, 1966. doi:10.1137/0114052.

[IL95] Neil Immerman and Susan Landau. The complexity of iterated multiplication. *Information and Computation*, 116(1):103–116, 1995. doi:10.1006/INCO.1995.1007.

[IMR80] Oscar H. Ibarra, Shlomo Moran, and Louis E. Rosier. A note on the parallel complexity of computing the rank of order n matrices. *Information Processing Letters*, 11(4/5):162, 1980. doi:10.1016/0020-0190(80)90042-3.

[IS99a] Balázs Imreh and Magnus Steinby. Directable nondeterministic automata. *Acta Cybernetica*, 14(1):105–115, 1999.

[IS99b] Balázs Imreh and Magnus Steinby. Directable nondeterministic automata. *Acta Cybernetica*, 14(1):105–115, 1999.

[IS08] Masami Ito and Kayoko Shikishima-Tsuji. Shortest directing words of nondeterministic directable automata. *Discrete Mathematics*, 308(21):4900–4905, 2008. doi:10.1016/J.DISC.2007.09.010.

[Jac77] Gérard Jacob. Un algorithme calculant le cardinal, fini ou infini, des demi-groupes de matrices. *Theoretical Computer Science*, 5(2):183–204, 1977. doi:10.1016/0304-3975(77)90006-8.

[Jam78] M. James. The generalised inverse. *The Mathematical Gazette*, 62:109–114, 1978.

[JJUW11] Rahul Jain, Zhengfeng Ji, Sarvagya Upadhyay, and John Watrous. QIP = PSPACE. *Journal of the ACM*, 58(6):30:1–30:27, 2011. doi:10.1145/2049697.2049704.

[JL76] Neil D. Jones and William T. Laaser. Complete problems for deterministic polynomial time. *Theoretical Computer Science*, 3(1):105–117, 1976. doi:10.1016/0304-3975(76)90068-2.

[JLL76] Neil D. Jones, Y. Edmund Lien, and William T. Laaser. New problems complete for nondeterministic loc space. *Mathematical Systems Theory*, 10:1–17, 1976. doi:10.1007/BF01683259.

[Kar72] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller, James W. Thatcher, and Jean D. Bohlinger, editors, *Complexity of Computer Computations: Proceedings of a symposium on the Complexity of Computer Computations, held March 20–22, 1972*, pages 85–103. Springer US, Boston, MA, 1972.

[Kar01] Jarkko Kari. A counter example to a conjecture concerning synchronizing words in finite automata. *Bulletin of the EATCS*, 73:146, 2001.

[Kar17] Nasim Karimi. Reaching the minimum ideal in a finite semigroup. *Semigroup Forum*, 94(2):390–425, 2017. doi:10.1007/s00233-015-9770-9.

[Kie20] Stefan Kiefer. Notes on equivalence and minimization of weighted automata. https://arxiv.org/abs/2009.01217, 2020. arXiv:arXiv:2009.01217.

[KL86] R. Kannan and R. J. Lipton. Polynomial-time algorithm for the orbit problem. *Journal of the ACM*, 33(4):808–821, 1986. doi:10.1145/6490.6496.

[KM21] Stefan Kiefer and Corto N. Mascle. On nonnegative integer matrices and short killing words. *SIAM Journal on Discrete Mathematics*, 35(2):1252–1267, 2021. doi:10.1137/19M1250893.

[KMT17] Markus Krötzsch, Tomáš Masopust, and Michaël Thomazo. Complexity of universality and related problems for partially ordered NFAs. *Information and Computation*, 255:177–192, 2017. doi:10.1016/J.IC.2017.06.004.

[KMW17] Stefan Kiefer, Ines Marusic, and James Worrell. Minimisation of multiplicity tree automata. *Logical Methods in Computer Science*, 13(1), 2017. doi:10.23638/LMCS-13(1:16)2017.

[Koz77] Dexter Kozen. Lower bounds for natural proof systems. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science*, pages 254–266. IEEE Computer Society, 1977. doi:10.1109/SFCS.1977.16.

[KRS87] A.A. Klyachko, I.K. Rystsov, and M.A. Spivak. An extremal combinatorial problem associated with the bound on the length of a synchronizing word in an automaton. *Cybernetics*, 23(2):165–171, 1987. doi:10.1007/BF01071771.

[KRV19] Jarkko Kari, Andrew Ryzhikov, and Anton Varonka. Words of minimum rank in deterministic finite automata. In Piotrek Hofman and Michal Skrzypczak, editors, *Developments in Language Theory – 23rd International Conference, DLT 2019, Warsaw, Poland, August 5-9, 2019, Proceedings*, volume 11647 of *Lecture Notes in Computer Science*, pages 74–87. Springer, 2019. doi:10.1007/978-3-030-24886-4_5.

[KV21] Jarkko Kari and Mikhail V. Volkov. Černý's conjecture and the road colouring problem. In *Handbook of Automata Theory*, pages 525–565. EMS Press, 2021. doi:10.4171/AUTOMATA-1/15.

[KW19a] Stefan Kiefer and Cas Widdershoven. Efficient analysis of unambiguous automata using matrix semigroup techniques. *CoRR*, abs/1906.10093, 2019. arXiv:1906.10093.

[KW19b] Stefan Kiefer and Cas Widdershoven. Efficient analysis of unambiguous automata using matrix semigroup techniques. In Peter Rossmanith, Pinar Heggernes, and Joost-Pieter Katoen, editors, *44th International Symposium on Mathematical Foundations of Computer Science, MFCS 2019, August 26-30, 2019, Aachen, Germany*, volume 138 of *LIPIcs*, pages 82:1–82:13. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019. doi:10.4230/LIPICS.MFCS.2019.82.

[LM21] Douglas A. Lind and Brian Marcus. *An introduction to symbolic dynamics and coding*. Cambridge university press, 2021. doi:10.1017/9781108899727.

[LP04] Alexei Lisitsa and Igor Potapov. Membership and reachability problems for row-monomial transformations. In Jiří Fiala, Václav Koubek, and Jan Kratochvíl, editors, *Mathematical Foundations of Computer Science 2004*, pages 623–634, Berlin, Heidelberg, 2004. Springer Berlin Heidelberg.

[Mar04] Nicolas Markey. Past is for free: on the complexity of verifying linear temporal properties with past. *Acta Informatica*, 40:431–458, 2004.

[Mar09] Pavel Martyugin. Complexity of problems concerning reset words for some partial cases of automata. *Acta Cybernetica*, 19(2):517–536, 2009.

[Mar10] Pavel V. Martyugin. Complexity of problems concerning carefully synchronizing words for PFA and directing words for NFA. In Ablayev and Mayr [AM10], pages 288–302.

[Mar12] Pavel V. Martyugin. Synchronization of automata with one undefined or ambiguous transition. In Moreira and Reis [MR12], pages 278–288.

[Mar14a] Pavel Martyugin. Computational complexity of certain problems related to carefully synchronizing words for partial automata and directing words for nondeterministic automata. *Theory Comput. Syst.*, 54(2):293–304, 2014. doi:10.1007/S00224-013-9516-6.

[Mar14b] Pavel Martyugin. Computational complexity of certain problems related to carefully synchronizing words for partial automata and directing words for nondeterministic automata. *Theory of Computing Systems*, 54(2):293–304, 2014.

[MK21] Tomáš Masopust and Markus Krötzsch. Partially Ordered Automata and Piecewise Testability. *Logical Methods in Computer Science*, 17(2):14:1–14:36, 2021. doi:10.23638/LMCS-17(2:14)2021.

[MOF08] Carlos Martín-Vide, Friedrich Otto, and Henning Fernau, editors. *Language and Automata Theory and Applications, Second International Conference, LATA 2008, Tarragona, Spain, March 13-19, 2008. Revised Papers*, volume 5196 of *Lecture Notes in Computer Science*. Springer, 2008.

[MP90] Zohar Manna and Amir Pnueli. A hierarchy of temporal properties. In Cynthia Dwork, editor, *Proceedings of the Ninth Annual ACM Symposium on Principles of Distributed Computing, Quebec City, Quebec, Canada, August 22-24, 1990*, pages 377–410. ACM, 1990. doi:10.1145/93385.93442.

[MR12] Nelma Moreira and Rogério Reis, editors. *Implementation and Application of Automata - 17th International Conference, CIAA 2012, Porto, Portugal, July 17-20, 2012. Proceedings*, volume 7381 of *Lecture Notes in Computer Science*. Springer, 2012.

[MS77] Arnaldo Mandel and Imre Simon. On finite semigroups of matrices. *Theoretical Computer Science*, 5(2):101–111, 1977. doi:10.1016/0304-3975(77)90001-9.

[Nat86] Balas K. Natarajan. An algorithmic approach to the automated design of parts orienters. In *27th Annual Symposium on Foundations of Computer Science, Toronto, Canada, 27-29 October 1986* [DBL86], pages 132–142.

[OPSPW19] Joël Ouaknine, Amaury Pouly, João Sousa-Pinto, and James Worrell. On the decidability of membership in matrix-exponential semigroups. *J. ACM*, 66(3), 2019. doi:10.1145/3286487.

[Pat70] Mike Paterson. Unsolvability in 3×3 matrices. *Studies in Applied Mathematics*, 49:105–107, 1970. doi:10.1002/sapm1970491105.

[PR20] Dominique Perrin and Andrew Ryzhikov. The Degree of a Finite Set of Words. In Nitin Saxena and Sunil Simon, editors, *40th IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2020)*, volume 182 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 54:1–54:16, Dagstuhl, Germany, 2020. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.FSTTCS.2020.54.

[Pro10] V. Yu. Protasov. Invariant functionals for random matrices. *Functional Analysis and Its Applications*, 44(3):230–233, 2010. doi:10.1007/s10688-010-0031-0.

[Pro21] Vladimir Yu. Protasov. Analytic methods for reachability problems. *Journal of Computer and System Sciences*, 120:1–13, 2021. doi:10.1016/J.JCSS.2021.02.007.

[PS09] Marko D. Petković and Predrag S. Stanimirović. Generalised matrix inversion is not harder than matrix multiplication. *Journal of Computational and Applied Mathematics*, 230:270–282, 2009.

[PS17] Igor Potapov and Pavel Semukhin. Decidability of the membership problem for 2×2 integer matrices. In Philip N. Klein, editor, *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2017, Barcelona, Spain, Hotel Porta Fira, January 16-19*, pages 170–186. SIAM, 2017. doi:10.1137/1.9781611974782.12.

[PV12] V Yu Protasov and AS Voynov. Sets of nonnegative matrices without positive products. *Linear Algebra and its Applications*, 437(3):749–765, 2012.

[PV17a] Vladimir Yu. Protasov and Andrey S. Voynov. Matrix semigroups with constant spectral radius. *Linear Algebra and its Applications*, 513:376–408, 2017.

[PV17b] Vladimir Yu. Protasov and Andrey S. Voynov. Matrix semigroups with constant spectral radius. *Linear Algebra and its Applications*, 513:376–408, 2017. doi:10.1016/j.laa.2016.10.013.

[RAH13] Charles R. Johnson Roger A. Horn. *Matrix analysis*. Cambridge University Press, 2nd edition, 2013. doi:10.1017/CBO9780511810817.

[Rei08] Omer Reingold. Undirected connectivity in log-space. *Journal of the ACM*, 55(4), 2008. doi:10.1145/1391289.1391291.

[RHK19] Peter Rossmanith, Pinar Heggernes, and Joost-Pieter Katoen, editors. *44th International Symposium on Mathematical Foundations of Computer Science, MFCS 2019, August 26-30, 2019, Aachen, Germany*, volume 138 of *LIPIcs*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019.

[RS14] Venkatesh Raman and S. P. Suresh, editors. *34th International Conference on Foundation of Software Technology and Theoretical Computer Science, FSTTCS 2014, December 15-17, 2014, New Delhi, India*, volume 29 of *LIPIcs*. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 2014.

[RS18] Andrew Ryzhikov and Anton Shemyakov. Subset synchronization in monotonic automata. *Fundamenta Informaticae*, 162(2-3):205–221, 2018.

[RSX12] Narad Rampersad, Jeffrey O. Shallit, and Zhi Xu. The computational complexity of universality problems for prefixes, suffixes, factors, and subwords of regular languages. *Fundam. Informaticae*, 116(1-4):223–236, 2012. doi:10.3233/FI-2012-680.

[Rys80] I. K. Rystsov. On minimizing the length of synchronizing words for finite automata. In *Theory of Designing of Computing Systems*, pages 75–82. Institute of Cybernetics of Ukrainian Acad. Sci., 1980. (in Russian).

[Rys83] Igor K. Rystsov. Polynomial complete problems in automata theory. *Information Processing Letters*, 16(3):147–151, 1983.

[Rys92] Igor Rystsov. Rank of a finite automaton. *Cybernetics and Systems Analysis*, 28(3):323–328, 1992. doi:10.1007/BF01125412.

[Rys97] Igor K. Rystsov. Reset words for commutative and solvable automata. *Theoretical Computer Science*, 172(1-2):273–279, 1997. doi:10.1016/S0304-3975(96)00136-3.

[Ryt86] Wojciech Rytter. The space complexity of the unique decipherability problem. *Information Processing Letters*, 23(1):1–3, 1986. doi:10.1016/0020-0190(86)90121-3.

[Ryz19a] Andrew Ryzhikov. Mortality and synchronization of unambiguous finite automata. In Robert Mercas and Daniel Reidenbach, editors, *Combinatorics on Words – 12th International Conference, WORDS 2019, Loughborough, UK, September 9-13, 2019, Proceedings*, volume 11682 of *Lecture Notes in Computer Science*, pages 299–311. Springer, 2019. doi:10.1007/978-3-030-28796-2_24.

[Ryz19b] Andrew Ryzhikov. Synchronization problems in automata without non-trivial cycles. *Theoretical Computer Science*, 787:77–88, 2019.

[Ryz24] Andrew Ryzhikov. On shortest products for nonnegative matrix mortality. In Laura Kovács and Ana Sokolova, editors, *Reachability Problems – 18th International Conference, RP 2024, Vienna, Austria, September 25-27, 2024, Proceedings*, volume 15050 of *Lecture Notes in Computer Science*, pages 104–119. Springer, 2024. doi:10.1007/978-3-031-72621-7_8.

[San04] Sven Sandberg. Homing and synchronizing sequences. In Broy et al. [BJK+05], pages 5–33.

[Sav70] Walter J. Savitch. Relationships between nondeterministic and deterministic tape complexities. *Journal of Computer and System Sciences*, 4(2):177–192, 1970.

[Sch70] Štefan Schwarz. On a sharp estimation in the theory of binary relations on a finite set. *Czechoslovak Mathematical Journal*, 20(4):703–714, 1970.

[Sen06] Eugene Seneta. *Non-negative matrices and Markov chains*. Springer Science & Business Media, 2006.

[Sip13] Michael Sipser. *Introduction to the Theory of Computation*. Course Technology, Boston, MA, third edition, 2013.

[Ste10] Benjamin Steinberg. A theory of transformation monoids: Combinatorics and representation theory. *The Electronic Journal of Combinatorics*, 17(1), 2010. doi:10.37236/436.

[SY10] Sujin Shin and Jisang Yoo. A note on the rank of semigroups. *Semigroup Forum*, 81(2):335–343, 2010. doi:10.1007/s00233-010-9225-2.

[Szy15] Marek Szykuła. Checking whether an automaton is monotonic is NP-complete. In Drewes [Dre15], pages 279–291.

[Tod90] Seinosuka Toda. On the complexity of topological sorting. *Information Processing Letters*, 35(5):229–233, 1990. doi:10.1016/0020-0190(90)90050-8.

[Tra07] Avraham Trakhtman. The Černý conjecture for aperiodic automata. *Discrete Mathematics and Theoretical Computer Science*, 9(2), 2007.

[TV19] Bianca Truthe and Mikhail V. Volkov. Journal of Automata, Languages and Combinatorics – Essays on the Černý Conjecture. https://www.jalc.de/issues/2019/issue_24_2-4/content.html, 2019. Accessed: 10/1/2020.

[TY15] Uraz Cengiz Türker and Hüsnü Yenigün. Complexities of some problems related to synchronizing, non-synchronizing and monotonic automata. *International Journal of Foundations of Computer Science*, 26(1):99–122, 2015.

[Vol08a] Mikhail V. Volkov. Synchronizing automata and the cerny conjecture. In Carlos Martín-Vide, Friedrich Otto, and Henning Fernau, editors, *Language and Automata Theory and Applications, Second International Conference, LATA 2008, Tarragona, Spain, March 13-19, 2008. Revised Papers*, volume 5196 of *Lecture Notes in Computer Science*, pages 11–27. Springer, 2008. doi:10.1007/978-3-540-88282-4_4.

[Vol08b] Mikhail V. Volkov. Synchronizing automata and the Černý conjecture. In Martín-Vide et al. [MOF08], pages 11–27.

[Vol22] Mikhail V. Volkov. Synchronization of finite automata. *Russian Mathematical Surveys*, 77(5):819–891, 2022. doi:10.4213/rm10005e.

[VP15] Andrey S. Voynov and Vladimir Yu. Protasov. Compact noncontraction semigroups of affine operators. *Sbornik: Mathematics*, 206(7):921, 2015. doi:10.1070/SM2015v206n07ABEH004483.

[VR15] Vojtech Vorel and Adam Roman. Parameterized complexity of synchronization and road coloring. *Discrete Mathematics and Theoretical Computer Science*, 17(1):283–306, 2015.

[Wol19] Petra Wolf. Synchronization under dynamic constraints. *CoRR*, abs/1910.01935, 2019. URL: http://arxiv.org/abs/1910.01935, arXiv:1910.01935.

[Wol20] Petra Wolf. Synchronization under dynamic constraints. In *40th IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2020)*, 2020.

[WZ23a] Yaokun Wu and Yinfeng Zhu. Primitivity and Hurwitz primitivity of nonnegative matrix tuples: A unified approach. *SIAM Journal on Matrix Analysis and Applications*, 44(1):196–211, 2023. doi:10.1137/22M1471535.

[WZ23b] Yaokun Wu and Yinfeng Zhu. Primitivity and Hurwitz Primitivity of Nonnegative Matrix Tuples: A Unified Approach. *SIAM Journal on Matrix Analysis and Applications*, 44(1):196–211, 2023. doi:10.1137/22M1471535.